

Åpne systemer, multi-agent-arkitektur, codec-teknologi og Claude Code

En gjennomarbeidet analyse av de fire teknologistakkene som definerer bygging av agentbaserte og medietunge systemer — med referansearkitekturer, sammenligningsgrunnlag, kodeeksempler og en prioritert innføringsplan.

DOKUMENT	Konsulentrapport, komplett utgave (30 sider)
VERSJON	1.0 — endelig
DATO	1. august 2026
MÅLGRUPPE	Teknologiledelse, arkitekter, tech leads og utviklere
OMFANG	Åpne standarder · Multi-agent-systemer · Video-, lyd- og datakodeker · Claude Code og integrasjoner · Tilgrensende teknologi, sikkerhet og regulering
KUNNSKAPS-GRUNNLAG	Primærspesifikasjoner, leverandørdokumentasjon og publisert benchmark-litteratur, verifisert per juli–august 2026

RAPPORTENS HOVEDPÅSTAND

De fire områdene i mandatet ser urelaterte ut, men styres av det **samme** underliggende mønsteret: verdien flytter seg fra implementasjonen til **grensesnittet**. En codec er en forhandlet kontrakt mellom koder og dekker; MCP og A2A er forhandlede kontrakter mellom modell og verktøy, og mellom agent og agent; Claude Codes utvidelsesprimitiver er forhandlede kontrakter mellom menneske og agent. Organisasjoner som behandler grensesnittvalg som en arkitekturbeslutning — ikke en implementasjonsdetalj — får lavere byttekostnad, raskere leveranse og målbart lavere driftsrisiko. Rapporten dokumenterer dette område for område og omsetter det i 12 konkrete anbefalinger (kapittel 26).

2026-07-28

GJELDENDE MCP-SPESIFIKASJON

150+

ORGANISASJONER BAK A2A

~30 %

BITRATEBESPARELSE: AV2 MOT AV1

≈15×

TOKENFORBRUK, MULTI-AGENT VS. CHAT

Navigasjon

Innholdsfortegnelse

Innledende	3
— Sammen drag, funn og leserveiledning	3
Del I · Åpne systemer og arkitekturer	4–8
1 Hva «åpent» faktisk betyr: fire uavhengige dimensjoner	4
1.1 Åpenhetsmatrisen · 1.2 Modenhetsmodell for standarder · 1.3 Innlåsningsøkonomi og byttekostnad	
2 Standardlandskapet 2026: kartlegging og modenhetsvurdering	5
2.1 Agent- og verktøylaget · 2.2 Kjørings-, transport- og observabilitetslaget · 2.3 Identitet og forsyningskjede	
3 MCP 2026-07-28 i dybden: fra økt til statsløs kerne	6
3.1 Bruddendringene · 3.2 Multi Round-Trip Requests · 3.3 Autorisasjonsherding · 3.4 Migreringsoppskrift	
4 Referansearkitektur for en åpen agentplattform	7
4.1 Sjulagsmodellen · 4.2 Åtte designprinsipper · 4.3 Grensesnittkontrakter mellom lagene	
5 Implementeringsstrategi, anti-mønstre og beste praksis	8
5.1 Femfasemodellen · 5.2 Syv anti-mønstre · 5.3 Innføringssjekkliste	
Del II · Multi-agent-systemer	9–14
6 Når multi-agent lønner seg — og når det ikke gjør det	9
7 Topologier: seks koordineringsmønstre med egenskapsprofil	10
8 Koordineringsmekanikk: dekomponering, kontrakter og terminering	11
9 Tekniske spesifikasjoner: A2A Agent Card, oppgavemaskin, meldingsskjema	12
10 Feilmodi, evaluering og observabilitet i produksjon	13
11 Fem gjennomarbeidede use cases fra reell drift	14
Del III · Codec-teknologier	15–20
12 Fundamentet: rate-distorsjon, BD-rate og kvalitetsmetriker	15
13 Videokodeker: full sammenligning fra H.264 til AV2	16
14 AV2 i dybden, enkodeøkosystem og bitrate-stigestrategi	17
15 Lydkodeker: Opus, xHE-AAC, LC3plus og tapsfrie formater	18
16 Nevrale codecs: RVQ-arkitektur, taletokens og produksjonsvurdering	19
17 Datakompresjon: zstd, brotli, kolonneformater og serialisering	20
Del IV · Claude Code og integrasjoner	21–26
18 Arkitektur, overflater og kontekstøkonomi	21
19 Utvidelsesprimitivene: hva du skal bruke når	22
20 Hooks i dybden: hendelser, kontrakt og styringsmønstre	23
21 Subagenter, skills og plugins: filformater og pakking	24
22 MCP-integrasjon og Agent SDK: egne servere og egne agenter	25
23 Produksjonssetting: CI/CD, sikkerhet, styring og kostnadskontroll	26
Del V · Øvrige relevante områder	27–28
24 Tilgrensende stakk: RAG, evaluering, isolering og forsyningskjede	27
25 Regulatorisk bilde og risikohåndtering i norsk/EØS-kontekst	28
Avslutning og vedlegg	29–30
26 Konklusjoner, 12 anbefalinger og treårig veikart	29
A–D Vedlegg: ordliste, kilder, sjekklister og metode	30

SLIK LESER DU RAPPORTEN

Hvert kapittel er selvstendig lesbart. **Har du 10 minutter:** les sammendraget (s. 3) og anbefalingene (s. 29). **Har du en time:** legg til de fire innledningskapitlene 1, 6, 12 og 18, som rammer inn hver del. **Skal du implementere:** kapitlene 5, 9, 14, 20–22 er skrevet som arbeidsdokumenter med kjørbare eksempler og sjekklister. Alle tall som er hentet fra en kilde, er merket i teksten og listet i vedlegg B; alle tall uten kildemerking er rapportens egne estimater og er merket som det.

Innledende

Sammendrag og hovedfunn

Rapporten dekker fire teknologiområder som i praksis møtes i den samme leveransen: en moderne, agentdrevet tjeneste som også håndterer media. Under følger de ti funnene som bærer resten av dokumentet.

0.1 Ti hovedfunn

TABELL 0.1 — HOVEDFUNN, KONSEKVENNS OG HVOR DET ER DOKUMENTERT

#	Funn	Konsekvens for beslutningstaker	Kapittel
1	«Åpen» er fire uavhengige egenskaper, ikke én.	Åpen kildekode uten åpen governance gir liten beskyttelse mot innlåsing. Vurder lisens, styring, spesifikasjon og dataformat hver for seg i anskaffelser.	1
2	MCP 2026-07-28 gjorde protokollen statsløs.	Håndtrykk og <code>Mcp-Session-Id</code> er borte. MCP-servere kan nå skaleres bak vanlig round-robin uten delt tilstand — men eldre klienter og servere brytes. Migrering er et prosjekt, ikke en oppgradering.	3
3	A2A har vunnet agent-til-agent-laget.	150+ organisasjoner, 1.0-spesifikasjon og innebygd støtte hos alle tre store skyleverandører. MCP og A2A er komplementære: MCP = agent ↔ verktøy, A2A = agent ↔ agent.	2, 9
4	Multi-agent koster ~15× et chattekall i tokenforbruk.	Mønsteret lønner seg kun ved parallelliserbare, bredde-første oppgaver med høy verdi per kjøring. Bruk beslutningsmatrisen i kapittel 6 før du bygger.	6
5	Orkestrator-arbeider er standardtopologien, men termineringslogikken er det som feiler.	De fleste produksjonsproblemer er ikke dårlige svar, men agenter som ikke vet når de er ferdige. Budsjetter og eksplisitte avslutningskriterier er obligatorisk.	7, 8, 10
6	AV2 (mai 2026) gir ~30 % mot AV1 — men uten maskinvare før 2027–28.	AV2 er en planleggingsbeslutning for 2027+, ikke en migrering for 2026. AV1 er nå det riktige standardvalget for ny distribusjon; H.264 forblir kompatibilitetsgulvet.	13, 14
7	Opus er fortsatt riktig standardvalg for sanntidslyd etter femten år.	Nevrale codecs vinner kun der båndbredden er den harde begrensningen og beregningskraft finnes. De er et forskningsvalg, ikke et driftsvalg — med ett unntak: taletokens til tale-til-tale-modeller.	15, 16
8	Zstandard har gjort kompresjonsvalg til et nivåvalg.	Ett bibliotek dekker nå spennet fra LZ4-hastighet til nær-xz-ratio. Standardiser på <code>zstd</code> med per-arbeidsmengde nivå og ordbøker; behold <code>brotili</code> kun for statisk web.	17
9	Claude Codes verdi ligger i primitivene, ikke i chatten.	CLAUDE.md, skills, subagenter, hooks, plugins og MCP er seks distinkte lag med hvert sitt bruksområde. Team som ikke skiller dem, ender med skjør konfigurasjon og uforutsigbar oppførsel.	19–22
10	Hooks er det eneste deterministiske kontrollpunktet.	Alt som <i>må</i> skje (formatering, hemmelighetsskanning, blokkering av farlige kommandoer) hører hjemme i en hook — ikke i en instruksjon modellen kan velge å ignorere. <code>PreToolUse</code> er sikkerhetsporten.	20, 23

0.2 Hvordan de fire områdene henger sammen

Mandatet ser ut som fire separate temaer. I praksis er de fire lag i samme leveranse, og de deler tre egenskaper som gjør at beslutningene må tas sammen:

- **Grensesnittet er produktet.** En codec er en kontrakt mellom koder og dekode som overlever begge implementasjonene. MCP og A2A er nøyaktig det samme for agenter. Når kontrakten er åpen og stabil, kan implementasjonen skiftes ut uten at systemet må skrives om — dette er hele argumentet for åpne systemer, uttrykt i tre ulike domener.
- **Asymmetrisk kostnad.** I både videokoding og agentkjøring betaler man mye én gang (enkoding, planlegging) for å spare mange ganger (dekoding, gjentatt kjøring). Arkitektur som ikke utnytter asymmetrien — re-enkoding per forespørsel, agenter som gjentar samme utforskning — brenner penger uten kvalitetsgevinst.
- **Kvalitet må måles med en proxy.** VMAF modellerer menneskelig oppfatning; en LLM-som-dommer modellerer menneskelig vurdering. Begge kan optimaliseres mot til systemet ser bra ut på metrikken og dårlig ut for brukeren. Disiplinen er den samme: hold en menneskeverifisert kontrollmengde utenfor optimaliseringssøylen.

DET VIKTIGSTE ENKELTRÅDET

Hvis organisasjonen bare gjør én ting etter denne rapporten: **etabler en eksplisitt kontraktgrense mellom agentlaget og verktøylaget (MCP), og mellom agenter (A2A), før dere bygger den andre integrasjonen.** Kostnaden ved å innføre grensen i etterkant vokser superlineært med antall integrasjoner, mens gevinsten — utskiftbare modeller, testbare verktøy, reviderbar tilgangsstyring — er tilgjengelig fra dag én. Alt annet i rapporten er lettere å reversere enn denne beslutningen.

0.3 Avgrensning og kildegrunnlag

Rapporten er beslutningsgrunnlag, ikke produktkatalog. Tre avgrensninger gjelder: **(i)** kodek-tall er publiserte referansemålinger på standardiserte testsett — bruk dem til å rangere alternativer, ikke til å budsjettere kapasitet; **(ii)** agentøkonomiske tall varierer med modell og oppgavetype i en størrelsesorden, og oppgis derfor som forholdstall; **(iii)** kapittel 25 er en teknisk lesning av regelverket, ikke juridisk rådgivning.

Del I · Kapittel 1

Hva «åpent» faktisk betyr

«Åpen» brukes som ett ord om fire uavhengige egenskaper. Systemer som er åpne langs én akse og lukket langs de tre andre er vanlige — og de gir nesten ingen av gevinstene man kjøper åpenhet for.

1.1 Åpenhetsmatrisen: fire akser som må vurderes hver for seg

Anskaffelsesprosesser spør typisk «er det åpen kildekode?» og stopper der. Det er feil spørsmål alene, fordi kildekode tilgang ikke i seg selv gir rett til å påvirke retningen, garanterer ikke at et alternativ kan implementeres, og sier ingenting om hvorvidt dataene dine kan tas ut igjen. De fire aksene under er ortogonale: et system kan skåre høyt på hvilken som helst og lavt på resten.

TABELL 1.1 — DE FIRE ÅPENHETSAKSENE, HVA DE FAKTISK BESKYTTER MOT, OG TYPISK FEILSLUTNING

Akse	Konkret spørsmål å stille	Hva den beskytter mot	Typisk feilslutning
Åpen kildekode	Kan vi lese, endre og kjøre koden selv, uten leverandøren?	Leverandørens død, prisøkninger, uakseptable produktveivalg	«Kildekoden er på GitHub» — men bygget krever en lukket tjeneste, eller lisensen er kildetilgjengelig og ikke fri (BSL, SSPL)
Åpen spesifikasjon	Finnes en normativ tekst som en tredjepart kan implementere mot?	Utskiftbarhet — muligheten til å bytte implementasjon uten å skrive om integrasjonen	Dokumentasjon forveksles med spesifikasjon. Uten samsvarstester (conformance suite) er «kompatibel» en påstand, ikke en egenskap
Åpen governance	Hvem kan si nei til en endring, og hvordan velges de?	Ensidige bruddendringer og «embrace–extend»-manøvrer	En stiftelse med én dominerende bidragsyter og betalte styreplasser er ikke reelt flerpartsstyrt
Åpne data og formater	Kan vi eksportere alt, i et format vi kan lese om ti år, uten leverandørens verktøy?	Datainnlåsing — den varigste og dyreste formen	Eksport finnes, men er ufullstendig (mangler metadata, relasjoner, historikk) eller uoverkommelig treg i praktisk skala

1.2 Modenhetsmodell: hvor mye vekt tåler en standard?

Å velge en standard er å inngå et veddemål om at den finnes om fem år. Nivåmodellen under brukes i rapporten til å klassifisere hver standard i kapittel 2, og gir en direkte oversettelse til hvor mye arkitektonisk vekt den kan bære.

TABELL 1.2 — MODENHETSNIVÅER FOR EN TEKNISK STANDARD

Nivå	Betegnelse	Objektive kriterier	Anbefalt arkitektonisk bruk
0	Forslag	Én implementasjon, ingen ekstern governance, uklar versjonering	Prototyper. Isoler bak eget grensesnitt slik at det kan rives ut på en dag
1	Framvoksende	2+ uavhengige implementasjoner, offentlig spesifikasjon, aktiv utvikling, hyppige brudd	Interne systemer. Forvent minst én migrering per år; budsjetter for den
2	Etablert	Flerpartsstyring, publisert livssyklus/avviklingspolicy, samsvarstester finnes	Trygt som integrasjonsgrense mellom team og mot leverandører
3	Infrastruktur	Uavhengig av opphavsorganisasjonen, implementert av konkurrenter, bakoverkompatibilitet håndhevet	Trygt som grunnmur. Kan legges i kontrakter og langsiktige plattformvalg
4	Universell	Antatt tilstede av økosystemet; å ikke støtte den er den bemerkelsesverdige beslutningen	Standardvalg. Avvik krever skriftlig begrunnelse

1.3 Innlåsningsøkonomi: hvorfor grensen må trekkes tidlig

Argumentet for åpenhet er ofte prinsipielt. Det bør være økonomisk. Byttekostnaden ved en lukket integrasjon vokser ikke lineært med antall integrasjoner, men med antall *koblingspunkter* — og de vokser kvadratisk når hver ny komponent integreres direkte mot alle de andre. Med n komponenter uten et felles kontraktlag nærmer antall integrasjonspaar seg $n(n-1)/2$; med et kontraktlag er det n .

TABELL 1.3 — KOBLINGSPUNKTER, DIREKTE VS. KONTRAKTLAG

Komponenter	Direkte (n^2)	Med kontrakt (n)	Faktor
4	6	4	1,5×
8	28	8	3,5×
15	105	15	7,0×
25	300	25	12,0×

PRAKTISK KONSEKVENS

Terskelen der et kontraktlag lønner seg, passeres typisk ved den **trede** integrasjonen — ikke ved den tiende. Fram til da føles direkte integrasjon raskere, og det er den også. Feilen er å bruke den opplevde hastigheten ved integrasjon 1–2 som argument for å utsette grensen til integrasjon 10, når ombyggingen koster 10–20× det den ville gjort tidlig.

Dette er den strukturelle begrunnelsen for hele Del I: MCP og A2A er ikke interessante fordi de er nye, men fordi de er det første bredt aksepterte kontraktlaget for henholdsvis verktøytilgang og agentsamhandling.

Del I · Kapittel 2

Standardlandskapet 2026

Kartlegging av de standardene som faktisk bærer vekt i en agentbasert plattform, klassifisert etter modenhetsmodellen i tabell 1.2 og gruppert etter hvilket lag de dekker.

TABELL 2.1 — STANDARDER SOM BÆRER VEKT I EN MODERNE AGENT- OG MEDIEPLATTFORM (MODENHET PER AUGUST 2026)

Standard	Lag	Governance	Nivå	Hva den løser	Praktisk merknad
MCP (2026-07-28)	Agent ↔ verktøy	MCP-prosjektet, flerpartsstyrt	3	Uniform tilgang for modeller til verktøy, ressurser og prompter over JSON-RPC	Statsløs kjerne fra 2026-07-28; eldre transport er avvirket
A2A 1.0.1	Agent ↔ agent	Linux Foundation	3	Oppdagelse, identitet og oppgaveutveksling mellom uavhengige agenter	150+ organisasjoner; innebygd hos AWS, Azure og Google
OpenAPI 3.1	Tjeneste-API	Linux Foundation	4	Beskrivelse av HTTP-API-er; kildeformat for verktøygenerering	Fortsatt det beste utgangspunktet for å autogenerere MCP-verktøy
JSON Schema 2020-12	Datakontrakt	OpenJS / IETF-arbeid	4	Validering av verktøyinnganger og strukturerte modellutdata	Underliggende for både MCP-verktøy og «structured outputs»
OpenTelemetry + GenAI semconv	Observabilitet	CNCF	3	Spor, metrikker og logger med felles semantikk for LLM-kall og agentsteg	GenAI-konvensjonene er ferskere enn kjernen; forvent justeringer
OCI Image & Distribution	Pakking	Open Container Initiative	4	Uforanderlige, innholdsadresserte artefakter og distribusjon av dem	Brukes også til å distribuere modeller, policyer og plugin-bunter
CloudEvents 1.0	Hendelser	CNCF	2	Felles konvolutt for hendelser på tvers av meglere og skyer	Naturlig bærer for agent-livssyklus hendelser til revisjonsspor
AsyncAPI 3.0	Hendelses-API	Linux Foundation	2	Beskrivelse av meldingsdrevne grensesnitt, motstykket til OpenAPI	Undervurdert der agenter konsumerer strømmer, ikke forespørsler
OAuth 2.1 + RFC 9207	Autorisasjon	IETF	4	Delegert tilgang; utstedervalidering hindrer mix-up-angrep	Nå påkrevd i MCPs autorisasjonsprofil
SPIFFE / SPIRE	Arbeidslast-identitet	CNCF	2	Kryptografisk identitet til prosesser uten delte hemmeligheter	Riktig grunnlag når agenter skal handle på egne vegne
SLSA v1.1 + Sigstore	Forsyningskjede	OpenSSF	2	Verifiserbar herkomst for bygg, inkludert plugin- og skill-bunter	Blir kritisk idet agenter installerer utvidelser selv
WASI 0.2 / Component Model	Isolert kjøring	Bytecode Alliance	2	Sandkasse med kapabilitetsbasert tilgang for uklarert kode	Lovende for verktøykjøring; økosystemet er fortsatt ujevnt
AV1 / AV2	Videokodek	AOMedia	4 / 1	Royaltyfri videokoding; AV2-spesifikasjon 1.0.0 kom 28. mai 2026	AV1 er modent; AV2 mangler maskinvare til 2027–28
Opus (RFC 6716)	Lydkodek	IETF / Xiph	4	Royaltyfri lav-latens lyd for tale og musikk i ett format	Obligatorisk i WebRTC; ingen reell konkurrent i sanntid

2.1 Agent- og verktøylaget: to protokoller med klar arbeidsdeling

Den vanligste misforståelsen er at MCP og A2A konkurrerer. De opererer på ulike akser. **MCP er vertikal**: den kobler én modell eller agent nedover til verktøy, filer og datakilder, og forutsetter at klienten er den handlende parten. **A2A er horisontal**: den kobler likeverdige, uavhengig eide agenter sammen, der ingen part kontrollerer den andres kjøring, og der oppgaver kan være langvarige og asynkrone. En agentplattform av en viss størrelse trenger begge, og de møtes i én komponent: agenten som eksponerer et A2A Agent Card utad, og bruker MCP innad for å faktisk utføre arbeidet.

2.2 Kjørings-, transport- og observabilitetslaget

Her er nyheten at det *ikke* er noen nyhet — og det er et godt tegn. OCI, OpenTelemetry og CloudEvents løser innpakking, innsyn og hendelsestransport for agentsystemer like godt som for alt annet. Rådet er derfor konservativt: **ikke innfør en agentspesifikk observabilitetsstakk**. Bruk OpenTelemetry med GenAI-konvensjonene, slik at agentspor havner i samme system som resten av driften og kan korreleres med forespørsler, feil og kostnad. Det eneste reelt nye behovet — sporing av *samtalestruktur*: hvilken agent kalte hvem, med hvilket budsjett — lar seg uttrykke som vanlige spans med attributter.

2.3 Identitet og forsyningskjede: det svakeste leddet

Dette laget er minst modent og har høyest risiko. Når agenter installerer egne utvidelser, henter MCP-servere fra et register og delegerer til tredjepartsagenter, er angrepsflaten et forsyningskjedeproblem — ikke et promptproblem. Tre tiltak har best kost/nytte: signér og verifiser alle plugin- og skill-bunter (Sigstore); gi hver agent egen arbeidslastidentitet framfor delt tjenestekonto (SPIFFE); og la tilgangsomfanget følge oppgaven, ikke agenten, slik at et kompromittert steg ikke arver alle rettighetene i systemet.

Del I · Kapittel 3

MCP 2026-07-28: fra økt til statsløs kjerne

Juli-utgaven er den største endringen i protokollens levetid. Den fjerner håndtrykket, gjør hver forespørsel selvbeskrivende og flytter MCP fra «langlivet forbindelse» til «vanlig web-trafikk». Det gjør skalering trivielt og migrering ikke-trivielt.

3.1 Endringene, og hva de koster

TABELL 3.1 — HOVEDENDRINGER I 2026-07-28 MED DRIFTSMESSIG OG MIGRERINGSMESSIG KONSEKVENS

Endring	Hva som faktisk skjer	Driftsgevinst	Migreringskostnad
Statsløs kjerne	<code>initialize/initialized</code> og <code>Mcp-Session-Id</code> er fjernet. Protokollversjon, klientidentitet og kapabiliteter følger hver forespørsel i <code>_meta</code> . Ny valgfri <code>server/discover</code> for kapabilitetsoppdagelse	Enhver forespørsel kan lande på enhver instans bak vanlig round-robin — ingen delt tilstand, ingen klebrige økter	Høy Bryter alle klienter og servere som antar økt
Multi Round-Trip Requests (SEP-2322)	Erstatter serverinitierte forespørsler som krevde åpen strøm. Server svarer <code>resultType: "input_required"</code> ; klienten kaller på nytt med <code>inputResponses</code>	Bekreftelser og oppfølgingsspørsmål midt i et kall uten vedvarende forbindelse	Middels Krever ny sløfye i klienten
Header-basert ruting (SEP-2243)	<code>Mcp-Method</code> og <code>Mcp-Name</code> må følge Streamable HTTP-forespørsler	Gatewayer, ratebegrensere og brannmurer kan rute og måle uten å parse JSON-kropp	Lav To headere
Cachebare listesvar (SEP-2549)	<code>tools/list</code> , <code>prompts/list</code> , <code>resources/list</code> og <code>resources/read</code> returnerer <code>ttlMs</code> og <code>cacheScope</code>	Færre unødvendige rundturer; merkbart ved mange verktøy	Lav Valgfri å utnytte
Autorisasjonsherding	RFC 9207-utstedervalidering påkrevd (SEP-2468); <code>application_type</code> i DCR (SEP-837); legitimasjon bundet til utstedende server (SEP-2352); CIMD erstatter DCR som standard	Lukker mix-up- og token-videresendingsangrep	Middels Berører identitetsintegrasjon
Extensions-rammeverk	Tasks, MCP Apps og Enterprise Managed Authorization formaliseres som utvidelser. Tasks flyttes til <code>io.modelcontextprotocol/tasks</code> med <code>tasks/get</code> og <code>tasks/update</code> (SEP-2663)	Kjernen holdes liten; utvidelser kan utvikles i eget tempo	Lav Opt-in
Avviklingspolicy	Alle funksjoner får livssyklusen Aktiv → Utfaset → Fjernet, med minst tolv måneder mellom utfasing og tidligste fjerning. Roots, Sampling og Logging er utfaset (SEP-2577), i tillegg til eldre HTTP+SSE-transport	Planleggbar migrering; slutt på overraskende brudd	Middels Krever avviklingsregister

3.2 Statsløst kall med bekreftelse midtveis (MRTR)

```
Mcp-Method: tools/call    Mcp-Name: deploy_service    ← gateway ruter/måler på headerne alene
{ "jsonrpc": "2.0", "id": 42, "method": "tools/call", "params": {
  "name": "deploy_service", "arguments": { "service": "checkout", "env": "prod" },
  "_meta": {
    ← erstatter håndtrykket: all kontekst følger hvert kall
    "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/client": { "name": "acme-agent", "version": "3.1.0" } } } }

— Serveren trenger bekreftelse, men holder ingen forbindelse åpen —
{ "jsonrpc": "2.0", "id": 42, "result": { "resultType": "input_required",
  "requests": [ { "id": "confirm-1", "type": "elicitation",
    "message": "Deploy checkout til prod?",
    "schema": { "type": "object", "properties": { "approved": { "type": "boolean" } } } } ] } }

— Klienten kaller på nytt med svaret — og kan treffe en helt annen instans —
{ "jsonrpc": "2.0", "id": 43, "method": "tools/call", "params": {
  "name": "deploy_service", "arguments": { "service": "checkout", "env": "prod" },
  "inputResponses": [ { "id": "confirm-1", "content": { "approved": true } } ] } }
```

3.3 Migreringsoppskrift

- Kartlegg først.** Alt som leser `Mcp-Session-Id`, bruker Roots/Sampling/Logging eller HTTP+SSE utgjør migreringsomfanget.
- Legg på headerne først.** De er bakoverkompatible og gir umiddelbar gevinst i gateway og logg.
- Gjør serveren tilstandsløs internt** før håndtrykket fjernes: flytt økt-tilstand ut, verifiser at to instanser kan betjene samme klient, og først da slå av økt-ID-en.
- Erstatt serverinitierte forespørsler med MRTR.** Eneste endring som krever ny klientlogikk — test den isolert.
- Slå på cache sist**, med korte `ttlMs`. Cache som skjuler verktøyendringer er en vond feilklasser å diagnostisere.

FALLGRUVE: TOLVMÅNEDERSVINDUET ER ET MINIMUM, IKKE EN PLAN

Den nye avviklingspolicyen garanterer minst tolv måneder fra utfasing til tidligste fjerning. Team som venter til måned elleve, oppdager at den utfasede funksjonen var vedv. inn i flere integrasjoner enn antatt — Roots, Sampling og Logging er nettopp slike: de brukes sjelden, men av kode ingen husker at finnes. Før avviklede funksjoner inn i et register med eier og frist ved første varsel, og la registeret være en fast post i arkitekturgjennomgangen.

Referansearkitektur for en åpen agentplattform

Sju lag med eksplisitte kontrakter mellom seg. Poenget med modellen er ikke lagene i seg selv, men at hver grense er en navngitt, testbar kontrakt — det er den egenskapen som gjør komponenter utskiftbare.

4.1 Sjulagsmodellen

7	Opplevelse	CLI, IDE, chat, web, planlagte kjøring. Kontrakt: sesjons- og hendelsesmodell. Skal kunne byttes uten at noe under lag 7 endres.
6	Orkestrering	Planlegging, dekomponering, delegering, budsjetter og terminering. Kontrakt: A2A utad, intern oppgavegraf innad.
5	Agentkjøring	Sløyfen: modellkall → verktøyvalg → observasjon → oppdatert tilstand. Kontrakt: modell-API og verktøyskjema. Her hører kontekstforvaltning hjemme.
4	Verktøy og kapabiliteter	Kontrakt: MCP . Alt agenten kan gjøre i verden, med skjema, beskrivelse og tilgangsomfang. Ingen direkte SDK-kall forbi dette laget.
3	Policy og identitet	Hvem er agenten, hva har den lov til, hvem godkjente det. Kontrakt: OAuth 2.1 / OIDC, SPIFFE, <i>utenformodellen</i> . policy-as-code. Håndheves
2	Isolering og kjøring	Sandkasse, ressursgrenser, nettverkspolicy. Kontrakt: OCI, WASI, mikro-VM. Antar at agenten kan gjøre feil ting.
1	Innsyn og bevis	Spør, metrikker, revisjonslogg, herkomst. Kontrakt: OpenTelemetry + GenAI semconv, CloudEvents, SLSA-attestasjoner.

Figur 4.1 — Lagene er nummerert nedenfra fordi avhengigheten går oppover: lag 1–3 må virke uten at lag 5–7 samarbeider.

4.2 Syv designprinsipper

TABELL 4.1 — DESIGNPRINSIPPER, BEGRUNNELSE OG OBSERVERBAR TEST

Prinsipp	Begrunnelse	Hvordan du tester at det holder
Modellen er en avhengighet, ikke plattformen	Modellskifte skjer oftere enn arkitekturskifte; leverandørspesifikke antakelser blir gjeld innen et år	Bytt modellleverandør i staging. Må mer enn konfigurasjon endres, er prinsippet brutt
Kapabiliteter passerer alltid lag 4	Direkte SDK-kall fra agentkoden omgår skjema, policy og logging på én gang	Statisk analyse: null nettverkskall utenfor MCP-klienten i agentmodulene
Autorisasjon håndheves utenfor modellen	En prompt-instruksjon er en anmodning, ikke en kontroll; injeksjon gjør den omgåelig	Be agenten gjøre noe forbudt. Blokkeringen skal skje i lag 3, ikke i modellsvaret
Kontekst er en budsjettett ressurs	Kontekstvinduet er den knappeste ressursen og styrer både kostnad og kvalitet	Mål tokens per oppgavetype. Uten måling finnes ingen styring
Delegering krever eksplisitt budsjett	En subagent uten grense på steg, tid og tokens er en åpen løkke i produksjon	Gi en oppgave uten slutttilstand. Systemet skal stoppe på budsjett, ikke tidsavbrudd
Idempotens på alt som skriver	Agenter prøver på nytt; retry uten idempotensnøkkel gir dobbeltposteringer	Kall samme verktøy to ganger med samme nøkkel. Effekten skal inntreffe én gang
Bevis, ikke bare logg	Revisjon krever hva som ble gjort, av hvem og med hvilken godkjenning	Rekonstruér en vilkårlig kjøring fra i går utelukkende fra lag 1

4.3 Kontraktene mellom lagene er leveransen

Den vanligste implementeringsfeilen er å bygge lagene, men ikke kontraktene: pene moduler som likevel kaller på tvers av alle grenser. Håndhev det som en arkitekturregel i CI — per lagovergang ett navngitt grensesnitt, ett skjema, ett sett samsvarstester, og null kall som hopper over et lag.

4.4 Konkrete standardvalg per lag

TABELL 4.2 — RAPPORTENS STANDARDVALG DER INGEN SÆRSKILTE KRAV TILSIER NOE ANNET

Lag	Standardvalg	Alternativ	Velg alternativet når...
6	Egen orkestrator på oppgavegraf, A2A utad	Rammeverk (LangGraph e.l.)	...teamet er lite og arbeidsflytene stabilt grafformede
4	MCP-servere per domene, ikke per system	Direkte funksjonskall	...alt kjører i én prosess og aldri skal deles
3	OIDC + policy-as-code (OPA/Cedar), SPIFFE	API-nøkler med omfang	...aldri. Akseptabelt kun i prototyper
2	Container, droppede kapabiliteter, nektet-nett som standard	Mikro-VM eller WASI	...agenten kjører kode den selv har skrevet
1	OpenTelemetry + GenAI semconv til eksisterende backend	Dedikert LLM-observabilitet	...du trenger promptdiffing i samme verktøy

Del I · Kapittel 5

Implementeringsstrategi og beste praksis

Femfasemodellen under er rekkefølgeavhengig. Den vanligste årsaken til at agentplattformer stopper opp i fase 3, er at fase 1 ble hoppet over.

5.1 Femfasemodellen

TABELL 5.1 — INNFØRINGSFASER MED UTGANGSKRITERIER

Fase	Navn	Varighet	Innhold	Utgangskriterium (målbart)
1	Grunnmur	4–6 uker	Lag 1–3: telemetri med GenAI-semantikk, arbeidslastidentitet, policyhåndheving, sandkasse. Ingen agentfunksjonalitet ennå	Et vilkårlig verktøykall kan spores ende-til-ende og knyttes til en identitet og en godkjenning
2	Første kontrakt	3–5 uker	Én MCP-server for det mest brukte interne systemet. Skjema, feilmeldinger, tilgangsomfang, samsvarstester	To ulike klienter bruker serveren uten serverendring. Feilmeldinger er handlingsbare for en modell
3	Én agent i drift	6–10 uker	Én smal, verdifull arbeidsflyt ende-til-ende med evaluering og kostnadsmåling fra dag én	Suksessrate og kostnad per kjøring er kjent og stabil over 200+ reelle kjøring
4	Bredde	løpende	Flere verktøy og flere arbeidsflyter på samme grunnmur. Plugin-/skill-distribusjon internt	Ny arbeidsflyt går i drift uten endring i lag 1–4. Marginalkostnaden faller
5	Samhandling	løpende	A2A utad mot andre team, leverandører eller kunder. Signerte Agent Cards, føderert identitet	En ekstern agent løser en reell oppgave uten særskilt tilpasningsarbeid

5.2 Syv anti-mønstre — med tidlige varselsignaler

TABELL 5.2 — ANTI-MØNSTRE, HVORFOR DE OPPSTÅR OG HVA DU SER FØRST

Anti-mønster	Hva som går galt	Tidlig varselsignal
Verktøypesplasjon	Alt eksponeres som verktøy fordi det er lett. Modellen må velge blant 200 alternativer og velger feil oftere enn den velger riktig	Verktøybeskrivelsene begynner å inneholde ord som «ikke bruk denne hvis...»
Prompt som policy	Sikkerhetsregler skrives i systemprompten. De holder til noen skriver noe som overstyres dem	Ordet «aldri» forekommer mer enn to ganger i systemprompten
Adapter-lasagne	Et eget abstraksjonslag over MCP «i tillegg vi bytter». Nå har du to kontrakter å vedlikeholde, og den ene er udokumentert	Ingen utenfor teamet klarer å legge til et verktøy uten hjelp
Evaluerer til slutt	Evaluerer utsettes til systemet «er ferdig». Da finnes ingen baseline, og enhver endring er et sjansespill	Spørsmålet «ble det bedre?» besvares med anekdoter
Ubudsjettert delegering	Subagenter kan spawn subagenter uten tak. Én uheldig oppgave koster mer enn hele måneden	Kostnad per kjøring har lang hale uten forklaring
Skjult tilstand i cache	Aggressiv caching av verktøyliste og ressurser gjør at agenten handler på et verdensbilde som ikke finnes lenger	Feil som «virker bare første gang etter deploy»
Kopiert leverandørsett	Et eksempeloppsett fra en blogg tas rett i produksjon, inkludert vide tilganger og manglende grenser	Ingen kan forklare hvorfor en gitt tillatelse står i konfigurasjonen

5.3 Innføringssjekkliste

Før første agent i produksjon

- Telemetri med GenAI-semantikk på plass, med kostnad per kjøring som førsteklases metrikk
- Egen identitet per agent; ingen delte tjenestekontoer
- Alle skrivende verktøy er idempotente og har dokumentert angreoperasjon
- Budsjett for steg, tid og tokens håndheves i kode, ikke i prompt
- Evalueringssett på minst 50 reelle oppgaver med fasit, holdt utenfor utviklingssløyfen

Før ekstern samhandling (A2A)

- Agent Card er signert, og signaturen verifiseres ved hver oppdagelse
- Tilgangsomfang følger oppgaven, ikke agenten
- Innkommende oppgaver behandles som uklart inndata gjennom hele kjeden
- Frakobling er testet: hva skjer når motparten er nede eller svarer feil?
- Revisjonssporet kan besvare «hvem ba om hva, når, og med hvilken godkjenning»

DEN ENE TESTEN SOM AVDEKKER MEST

Ta en tilfeldig kjøring fra forrige uke og prøv å svare på tre spørsmål utelukkende fra telemetri: *hva forsøkte agenten å oppnå, hvilke verktøy kalte den med hvilke argumenter, og hvorfor stoppet den?* Team som ikke kan svare på alle tre, har et innsynsproblem som vil dominere all feilsøking senere — og det er billigere å løse i fase 1 enn i fase 4.

Del II · Kapittel 6

Når multi-agent lønner seg — og når det ikke gjør det

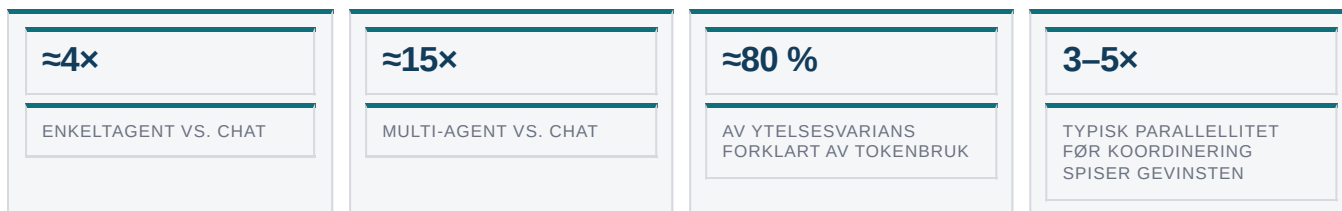
Multi-agent er ikke en oppgradering av enkeltagent. Det er en annen kostnads- og feilprofil, som gir gevinst på en smal, men verdifull klasse oppgaver — og som taper stort på alt utenfor den klassen.

6.1 Definisjon og avgrensning

Et multi-agent-system er her definert som: **flere LLM-drevne kjøring**er med hver sin kontekst, som koordinerer mot et felles mål gjennom eksplisitte meldinger eller delt tilstand. Tre ting følger av definisjonen, og de er alle konsekvensrike. For det første: hvis alle steg deler samme kontekstvindu, er det ikke multi-agent — det er en agent med god struktur, og den er nesten alltid billigere. For det andre: koordinering krever et *medium*, og valget av medium (meldinger, delt oppgavetavle, filsystem) bestemmer mer av systemets oppførsel enn valget av modell. For det tredje: kontekstisolasjon er både gevinsten og problemet — arbeideren slipper å se alt, men vet derfor heller ikke hva de andre gjør.

6.2 Tokenøkonomien: hvorfor terskelen ligger høyt

Publiserte driftserfaringer fra produksjonssatte forskningsagenter gir en nyttig tommelfingerregel: en enkeltagent-interaksjon bruker rundt 4× tokens av en vanlig chatsamtale, mens et multi-agent-oppsett bruker rundt 15×. Tokenforbruk alene forklarte omtrent 80 % av variansen i ytelse på oppgavene deres. Det er et tveegget funn: det forklarer hvorfor multi-agent virker, og samtidig hvorfor det er dyrt — mye av gevinsten kommer rett og slett av at systemet får bruke mer beregning på problemet.



Den praktiske konsekvensen er at multi-agent må rettferdiggjøres *per oppgavetype*, ikke per plattform. En oppgave der svaret er verdt 50 kroner, tåler ikke en arkitektur som koster 15× av 3 kroner. En oppgave der et godt svar sparer en fagperson for fire timer, gjør det lett.

6.3 Beslutningsmatrise

TABELL 6.1 — EGENSKAPER VED OPPGAVEN SOM AVGJØR ARKITEKTURVALGET

Egenskap ved oppgaven	Taler for enkeltagent	Taler for multi-agent	Hvorfor
Struktur	Sekvensiell, hvert steg avhenger av forrige	Bredde-først: mange uavhengige spor som kan utforskes samtidig	Parallellitet er hele gevinsten. Uten den betaler du koordinering for ingenting
Kontekstbehov	Alt må ses samlet for å gi mening	Delproblemene kan løses med lokal kontekst	Isolasjon fungerer bare når delene faktisk er separerbare
Verdi per kjøring	Lav til middels	Høy — timer spart eller høy risiko unngått	15× tokenkostnad krever tilsvarende utbytte
Verifiserbarhet	Vanskelig å sjekke uten menneske	Delresultater kan valideres maskinelt	Uten verifikasjon forsterker parallellitet feil i stedet for å fange dem
Latenstoleranse	Svar forventes i sekunder	Minutter er akseptabelt	Koordinering, syntese og retry koster veggklokke
Feilkostnad	Høy, og feil er vanskelig å oppdage	Moderat, eller feil fanges i validering	Flere agenter gir flere steder feil kan oppstå og forplante seg

REGELEN SOM SPARER MEST PENGER

Bygg enkeltagenten først, mål den, og innfør multi-agent bare der målingen viser at enkeltagenten treffer en reell grense. I praksis er «grensen» nesten alltid én av to: kontekstvinduet fylles av materiale agenten må se samtidig, eller veggklokketiden domineres av uavhengige oppslag som kunne gått parallelt. Er ingen av de to til stede, vil multi-agent gi dårligere resultat til høyere pris — og det er den vanligste utfallet av å begynne med arkitekturen i stedet for målingen.

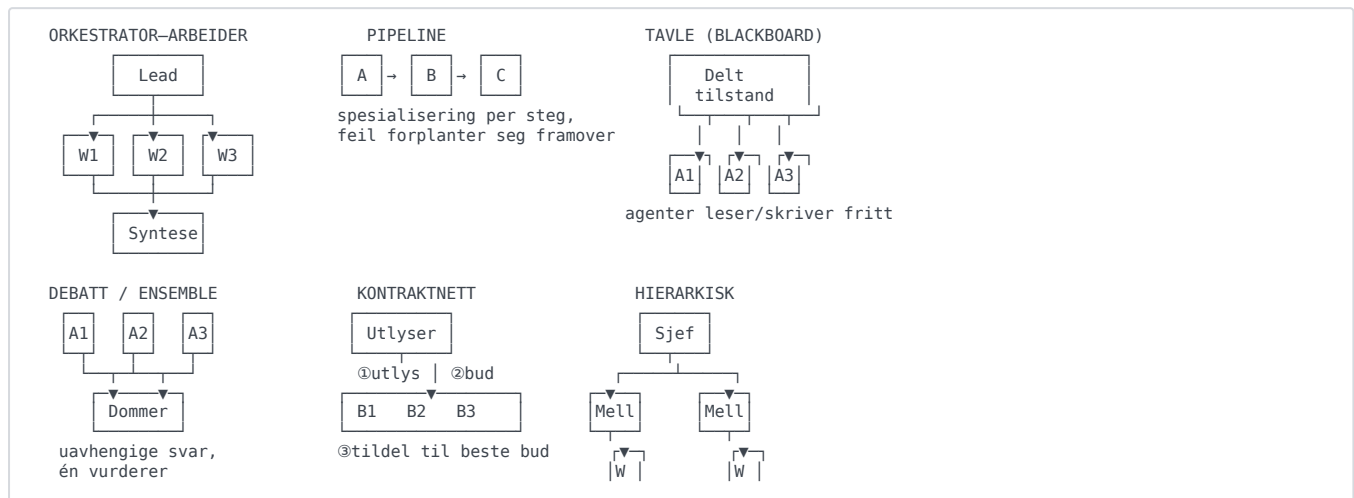
6.4 Tre gevinster som ikke handler om ytelse

Multi-agent velges av og til med rette selv når enkeltagenten ville løst oppgaven, fordi strukturen gir egenskaper som er vanskelige å oppnå på annen måte: (i) **tilgangsseparasjon** — en arbeider som bare har lesetilgang kan trygt sendes inn i uklart materiale, mens orkestratoren beholder skriverettighetene; (ii) **reviderbarhet** — når delegeringen er en eksplisitt melding med mandat og budsjett, får du et revisjonsspor som er lesbart for mennesker; og (iii) **uavhengig eierskap** — to team kan levere hver sin agent med hvert sitt utgivelsestempo, forutsatt at kontrakten mellom dem er reell. Disse tre er legitime grunner. «Det føles mer avansert» er det ikke.

Del II · Kapittel 7

Seks koordineringstopologier

Topologien bestemmer hvordan informasjon flyter, hvor feil oppstår og hva som er dyrt. De fleste produksjonssystemer er orkestrator–arbeider med ett eller to elementer lånt fra de øvrige.



Figur 7.1 — De seks topologiene. Pilene viser informasjonsflyt, ikke kontrollflyt; i praksis følger kontroll ofte motsatt vei.

TABELL 7.1 — EGENSKAPSPROFIL PER TOPOLOGI

Topologi	Bruk den når	Token-kostnad	Latens	Dominerende feilmodus	Nødvendig mottiltak
Orkestrator–arbeider	Bredde-først utforskning der delresultater kan slås sammen	Høy	Middels	Overlappende arbeid: to arbeidere gjør det samme	Presise, ikke-overlappende mandat med eksplisitt avgrensning
Pipeline	Fast rekkefølge med tydelig spesialisering per steg	Lav	Høy	Feilforplantning: steg 2 bygger tillitsfullt på feil fra steg 1	Validering mellom hvert steg; steg som kan avvise inndata
Tavle	Delproblemer oppdages underveis og er innbyrdes avhengige	Middels	Middels	Kappløp og skriveoverskriving på delt tilstand	Optimistisk låsing, versjonerte innslag, én skriver per nøkkel
Debatt/ensemble	Høy feilkostnad og ingen maskinell fasit	Svært høy	Middels	Korrelert feil: alle tar feil på samme måte	Varier prompt, modell og kilde per deltaker — ikke bare temperatur
Kontraktnett	Heterogene agenter med ulik kompetanse og kapasitet	Middels	Høy	Budinflasjon: alle hevder å kunne alt	Bud må være verifiserbare mot historikk, ikke selverklærte
Hierarkisk	Dyp oppgavestruktur som overstiger én agents kontekst	Høy	Høy	Tap i oversettelsen nedover og sammendragstap oppover	Maks to nivåer i praksis; strukturerte mandat- og rapportskjema

7.1 Hvorfor orkestrator–arbeider dominerer

Mønsteret vinner fordi det er det eneste som gir **parallelletet uten delt tilstand**. Arbeiderne trenger ikke å vite om hverandre; de trenger bare et presist mandat og en måte å levere tilbake på. Det gjør feilsøking håndterbar (hver arbeider kan kjøres alene med sitt mandat), kostnaden forutsigbar (n arbeidere $\times$ budsjett) og delegeringen reviderbar.

Den avgjørende designdetaljen er **mandatets presisjon**. Et vagt mandat («undersøk leverandørmarkedet») gir overlappende arbeid, og syntesen får tre varianter av samme informasjon framfor tre komplementære biter. Et godt mandat har alltid fire elementer: *mål*, *avgrensning mot søskenes områder*, *leveranseformat* og *budsjett*. Mandatgenerering er kjernearbeidet i orkestratoren — ikke oppgavelisten.

HYBRIDEN SOM FUNGERER BEST I PRAKSIS

Orkestrator–arbeider for utforskningsfasen, pipeline for produksjonsfasen. Bredden trenger parallelletet og tåler støy; produksjonen trenger rekkefølge og validering. Systemer som bruker én topologi hele veien, betaler enten for mye (parallellell produksjon som må samordnes) eller for lite (sekvensiell utforskning som tar timer).

7.2 Topologivalg som funksjon av oppgaven

To spørsmål avgjør. **Er delproblemene kjent på forhånd?** Er de det, står valget mellom orkestrator–arbeider (uavhengige deler) og pipeline (avhengige deler i fast rekkefølge); er de det ikke, trengs en tavle eller en orkestrator som replanlegger mellom runder — det siste er enklere å drifte, fordi det beholder én skriver. **Finnes en maskinell fasit?** Finnes den, er validering billigere enn konsensus. Finnes den ikke, og feilkostnaden er høy, er ensemble eneste topologi som reduserer risikoen for systematisk feil.

Del II · Kapittel 8

Koordineringsmekanikk

Fem mekanismer avgjør om et multi-agent-system er driftbart: dekomponering, mandatkontrakten, kontekst- og minnehåndtering, konfliktløsning og terminering. Den siste er den som oftest mangler.

8.1 Dekomponering: den viktigste enkeltoperasjonen

Kvaliteten på sluttresultatet bestemmes i stor grad før noen arbeider starter. Tre dekomponeringsstrategier dekker de fleste tilfeller: **etter dimensjon** (samme spørsmål stilt mot ulike kilder, markeder eller tidsrom), **etter fase** (innsamling, analyse, verifikasjon, syntese) og **etter hypotese** (hver arbeider forfølger én mulig forklaring til den holder eller faller). Hypotesedekomponering er den mest undervurderte: den gir naturlig ikke-overlappende mandat, og den produserer negative resultater — hvilke forklaringer som *ikke* holder — som er nettopp det syntese-teget trenger for å konkludere med trygghet.

Et praktisk mål på dekomponeringskvalitet: **andelen av arbeidernes leveranser som faktisk brukes i syntesen**. Ligger den under omtrent halvparten, er mandatene enten overlappende eller dårlig rettet, og pengene går til arbeid som kastes. Målet er lett å instrumentere — merk hvilke delresultater syntese-teget siterer — og er mer styrende enn suksessraten på sluttresultatet, fordi det peker på *hvor* i kjeden tapet oppstår.

8.2 Mandatkontrakten

Delegering bør være en strukturert melding, ikke fri tekst. Et minimum som fungerer:

```
{
  "task_id": "res-7f3a-04",
  "objective": "Fastslå om leverandør X har dokumentert ISO 27001-sertifisering gyldig i 2026",
  "scope_in": ["offentlige registre", "leverandørens egne publikasjoner", "sertifiseringsorgan"],
  "scope_out": ["pris og kommersielle vilkår", "andre leverandører"], ← hindrer overlapp med søsken
  "deliverable": { "format": "json",
    "schema": { "certified": "boolean", "valid_until": "date|null",
      "evidence_urls": "string[]", "confidence": "low|medium|high" } },
  "budget": { "max_tool_calls": 12, "max_tokens": 60000, "max_wall_clock_s": 180 },
  "on_insufficient_evidence": "return confidence=low with what was found; do NOT infer",
  "parent_context_digest": "Anskaffelse av driftsleverandør; sertifisering er et absolutt krav."
}
```

De to feltene som gjør mest forskjell, er `scope_out` (eneste effektive middel mot overlapp mellom søsken) og `on_insufficient_evidence` (eneste effektive middel mot at en arbeider som ikke finner svaret, gjetter — den mest kostbare feilen i systemet, fordi den er usynlig for syntese-teget).

8.3 Kontekst, minne og hva som faktisk skal deles

TABELL 8.1 — FIRE MINNETYPER, LEVETID OG RIKTIG LAGRINGSSTED

Type	Innhold	Levetid	Riktig sted, og typisk feil
Arbeidsminne	Aktiv kontekst i pågående kjøring	Ett steg	Kontekstvinduet. Feil: Å la det vokse ubegrenset i stedet for å komprimere ved terskel
Oppgaveminne	Delresultater, hva som er prøvd, hva som feilet	Én oppgave	Ekstern butikk med oppgave-ID. Feil: Å bare ha det i orkestratorens kontekst, som da må vokse med antall arbeidere
Erfaringsminne	Lærdom på tvers av kjøringer: bygghendelser, kjente fallgruver	Uker–måneder	Versjonert fil eller database med eier. Feil: Å la agenten skrive fritt, uten gjennomgang — minnet forgiftes av én dårlig kjøring
Delt tilstand	Det arbeiderne må se av hverandres arbeid	Én oppgave	Eksplisitt og minimal, med én skriver per nøkkel. Feil: full deling, som gjenoppretter alt man ville unngå med isolasjon

8.4 Konfliktløsning og terminering

Når to arbeidere leverer motstridende svar, må systemet ha en forhåndsbestemt regel. Å overlate det til syntesemodellen «å vurdere» gir ikke-reproduserbar oppførsel. Fire prioriterte regler dekker praktisk talt alt: **(1) bevisstyrke** — primærkilde slår sekundærkilde; **(2) ferskhets** — nyere slår eldre når spørsmålet er tidsavhengig; **(3) eskalering** — uløst motstrid sendes til menneske med begge svar; **(4) eksplisitt uenighet** — konklusjonen rapporterer at kildene spriker framfor å velge. Den siste er undervurdert og ofte det ærligste resultatet.

TERMINERING ER DEN VANLIGSTE PRODUKSJONSFEILEN

Agenter feiler sjelden ved å svare feil. De feiler ved å *ikke bli ferdige*: en arbeider som fortsetter å lete etter et bevis som ikke finnes, en orkestrator som starter en ny runde fordi forrige var «nesten» god nok. Hvert nivå trenger tre uavhengige stopp-punkter: **budsjett** (steg, tokens, tid — håndhevet i kode), **tilstrekkelighet** (et eksplisitt kriterium formulert før kjøring), og **stagnasjon** (ingen ny informasjon på N steg → avslutt og rapporter det du har). Uten alle tre havner systemet på tidsavbrudd, og tidsavbrudd gir ingen brukbar delleveranse.

Del II · Kapittel 9

Tekniske spesifikasjoner

A2A-spesifikasjonen definerer tre ting et multi-agent-system trenger og som ellers må oppfinnes lokalt: en oppdagelsesmekanisme, en identitetsmodell og en oppgavelivssyklus.

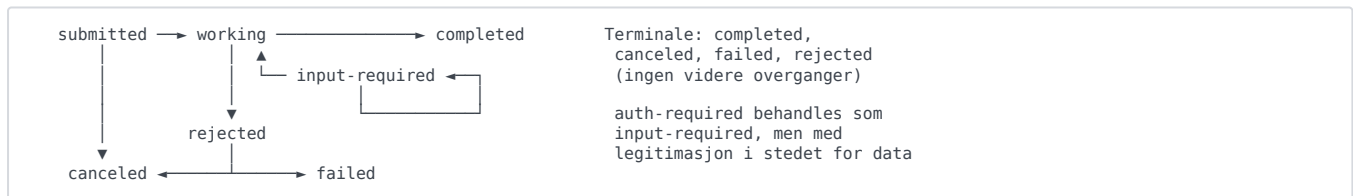
9.1 Agent Card: oppdagelse og signert identitet

Et Agent Card er et JSON-dokument publisert på et velkjent endepunkt, som beskriver hva agenten kan, hvordan man snakker med den, og hvordan man autentiserer. Signerte kort (innført med 1.0) gjør at mottakeren kan verifisere at kortet faktisk kommer fra den organisasjonen det utgir seg for — nødvendig i det øyeblikket agenter oppdager hverandre utenfor egen tillitssone.

```
GET https://agents.example.no/.well-known/agent-card.json
{
  "protocolVersion": "1.0.1",
  "name": "kontrakt-analyse",
  "description": "Analyserer leverandøravtaler mot intern policy og flagger avvik.",
  "url": "https://agents.example.no/a2a",
  "preferredTransport": "JSONRPC",
  "provider": { "organization": "Example AS", "url": "https://example.no" },
  "capabilities": { "streaming": true, "pushNotifications": true, "stateTransitionHistory": true },
  "defaultInputModes": [ "text/plain", "application/pdf" ],
  "defaultOutputModes": [ "application/json" ],
  "skills": [ {
    "id": "avviksanalyse",
    "name": "Avviksanalyse mot innkjøpspolicy",
    "description": "Leser en avtale og returnerer avvik med paragrafhenvising og alvorlighetsgrad.",
    "tags": [ "juridisk", "innkjøp" ],
    "examples": [ "Finn avvik i vedlagte rammeavtale mot policy v4.2" ],
    "outputModes": [ "application/json" ] },
    {
      "id": "tokenauth",
      "name": "OAuth2 token auth",
      "description": "Behandler OAuth2 token autentisering.",
      "tags": [ "auth", "token" ],
      "examples": [ "Autentiser med token fra kontrakt.read" ],
      "outputModes": [ "application/json" ] } ],
  "securitySchemes": { "oauth2": { "type": "oauth2", "flows": { "clientCredentials": {
    "tokenUrl": "https://id.example.no/token", "scopes": { "contract.read": "Les avtaler" } } } },
    "tokenUrl": "https://id.example.no/token", "scopes": { "contract.read": "Les avtaler" } } },
  "security": [ { "oauth2": [ "contract.read" ] } ],
  "signatures": [ { "protected": "eyJhbGciOiJIUzI1NiIsImtpZCI6Im...", ← JWS over kortet
    "signature": "MEUCIQD..." } ]
}
```

9.2 Oppgavelivssyklus

A2A modellerer arbeid som en *oppgave* med eksplisitte tilstander, ikke som en forespørsel med svar. Det er den riktige modellen når arbeid kan ta minutter til timer, må kunne følges og kan trenge inndata underveis.



TABELL 9.1 — TILSTANDER, ANSVAR OG TYPISK HÅNTERINGSFEIL

Tilstand	Betydning	Typisk feil hos klienten
submitted	Mottatt og køet, arbeid ikke startet	Å behandle mottak som bekreftelse på at oppgaven kan løses
working	Under arbeid; delresultater kan strømmes	Å polle med fast intervall i stedet for å bruke strøm eller push
input-required	Blokkert i påvente av avklaring fra kaller	Å ikke ha en vei tilbake til et menneske — oppgaven står til den utløper
auth-required	Blokkert i påvente av legitimasjon	Å be om nye rettigheter bredere enn oppgaven krever
completed	Ferdig med resultat (artefakter)	Å anta at «ferdig» betyr «riktig» — validering hører hjemme hos kaller
failed / rejected	Kunne ikke fullføres / ble avvist ved mottak	Å prøve på nytt uendret. rejected betyr som regel at forespørselen må endres
canceled	Avbrutt av kaller	Å ikke rydde opp i sideeffekter fra påbegynt arbeid

9.3 Arbeidsdelingen mot MCP

Én komponent bruker begge protokollene samtidig, og det er verdt å være presis om hvor grensen går. **Utad** eksponerer agenten et Agent Card og mottar oppgaver over A2A; den er da en likeverdig part som eier sin egen kjøring, og motparten har ingen innsikt i eller kontroll over hvordan arbeidet gjøres. **Innad** er agenten en MCP-klient som kaller verktøy for å faktisk utføre arbeidet; her er den den handlende parten, og verktøyene er passive. Systemer som prøver å bruke MCP til agent-til-agent-samhandling, ender med å gjenoppfinne oppgavelivssyklusen i tabell 9.1 — og systemer som bruker A2A til verktøykall, betaler oppdagelses- og identitetskostnad for noe som burde vært et funksjonsskall.

Del II · Kapittel 10

Feilmodi, evaluering og observabilitet

Multi-agent-systemer feiler på måter enkeltagenter ikke gjør. Feilene er systematiske, gjenkjennelige og — viktigst — mulige å instrumentere for.

10.1 Ti feilmodi

TABELL 10.1 — FEILMODI RANGERT ETTER HVOR OFTE DE DOMINERER PRODUKSJONSPROBLEMER

#	Feilmodus	Hvordan den ser ut	Mottiltak som virker
1	Manglende terminering	Kjøringer som treffer tidsavbrudd uten delleveranse; halen i kostnadsfordelingen er lang og uforklart	Tre uavhengige stopp: budsjett, tilstrekkelighetskriterium, stagnasjonsdeteksjon (kap. 8.4)
2	Overlappende arbeid	Flere arbeidere leverer samme funn; syntesen får bredde uten dybde	<code>scope_out</code> i mandatet; mål andel leveranser brukt i syntesen
3	Gjetting ved bevisunderskudd	Arbeider som ikke finner svaret, produserer et plausibelt ett — usynlig for syntesen	Eksplisitt <code>on_insufficient_evidence</code> ; krev konfidens og kildehenvisning i skjemaet
4	Kontekstforgiftning	En feilaktig påstand tidlig blir behandlet som etablert og siteres videre i alle senere steg	Skill mellom «observert» og «utledet» i tilstanden; la verifikasjonssteg kunne nedgradere påstander
5	Sammendragstap	Hvert nivå komprimerer, og det avgjørende forbeholdet forsvinner mellom nivå 2 og 3	Maks to nivåer; strukturerte rapportskjema der forbehold er et eget felt, ikke prosa
6	Korrelert feil i ensemble	Alle deltakerne tar feil på samme måte, og konsensus gir falsk trygghet	Varier modell, prompt og kildegrunnlag — ikke bare temperatur
7	Verktøyvalgkollaps	Agenten bruker ett generisk verktøy til alt fordi de spesifikke er dårlig beskrevet	Færre verktøy, bedre beskrivelser, eksempler i beskrivelsen; mål bruksfordelingen
8	Retry-storm	Feil nedstrøms utløser retry på alle nivåer samtidig; last mangedobles	Retry kun på ett nivå (nærmest feilen), med eksponentiell backoff og jitter
9	Ikke-idempotente sideeffekter	Retry etter delvis fullført skriving gir dobbeltposterings	Idempotensnøkkel per logisk operasjon, håndhevet i verktøylaget
10	Indirekte prompt-injeksjon	Hentet innhold inneholder instruksjoner agenten følger som om de kom fra brukeren	Behandle alt hentet innhold som data; håndhev autorisasjon i lag 3, aldri i prompten

10.2 Evaluering: tre nivåer som må finnes samtidig

Evaluering av agentsystemer feiler oftest fordi bare ett nivå måles. Tre nivåer er nødvendige, og de svarer på ulike spørsmål. **Utfallsevaluering** spør om sluttresultatet er riktig, og krever et fasitsett; det er det eneste nivået som måler det brukeren bryr seg om, men det sier ikke hvor det gikk galt. **Sporevaluering** (trajectory) spør om veien dit var fornuftig: ble riktige verktøy valgt, ble budsjettet brukt på relevante spor, gjentok agenten seg selv? Dette er nivået som gjør feilsøking mulig. **Komponentevaluering** spør om det enkelte verktøyet, retrieveren eller syntese-teget gjør jobben sin isolert — nødvendig for å vite om en regresjon skyldes modell, prompt eller infrastruktur.

For utfall uten maskinell fasit er en LLM-som-dommer det praktiske valget, med tre forbehold som er ufravikelige: dommeren må ha en *rubrikk* med konkrete kriterier og ikke bare «vurder kvaliteten»; dommeren må valideres mot menneskelige vurderinger på et utvalg, og samsvaret må rapporteres; og et kontrollsett må holdes utenfor all optimalisering, ellers måler man til slutt hvor godt systemet tilfredsstiller dommeren.

10.3 Observabilitet: hva som må instrumenteres

Per span (OpenTelemetry GenAI semconv)

- Agent-ID, forelder-ID, delegeringsdybde
- Modell, tokens inn/ut, cache-treff
- Verktøynavn, argumenthash, varighet, utfall
- Budsjett tildelt vs. brukt
- Termineringsårsak (fullført / budsjett / stagnasjon / feil)

Metrikker som faktisk styrer

- Kostnad per fullført oppgave (ikke per kall)
- Andel kjøring som ender i terminal tilstand med leveranse
- Andel arbeiderleveranser brukt i syntesen
- P50/P95 delegeringsdybde og veggklokkeid
- Termineringsårsak fordelt over tid

DEN MEST UNDERVURDERTE METRIKKEN

Fordelingen av termineringsårsaker forteller mer om systemets helse enn suksessraten. Et system der 70 % av kjøringene avsluttes på budsjett i stedet for på tilstrekkelighetskriterium, leverer systematisk halvferdig arbeid — og suksessraten kan likevel se akseptabel ut, fordi halvferdig arbeid ofte er godt nok til å bestå en grov vurdering. Følg denne fordelingen fra dag én; den avslører regresjoner uker før brukerne klager.

Del II · Kapittel 11

Fem use cases fra reell drift

Fem mønstre som er dokumentert i produksjon, med topologi, hva gevinsten faktisk kom fra, og den fallgruben som oftest velter dem.

11.1 Dyp research og kildesammenstilling

Topologi: orkestrator–arbeider, 3–8 arbeidere per runde, opptil tre runder. **Hvorfor det virker:** research er den arketypiske bredde-først-oppgaven — delspørsmålene er uavhengige, hver arbeider kan brenne kontekst på å lese mye og levere lite, og syntesen trenger bare destillatet. Det er også her de publiserte gevinstene er størst: et orkestrator–arbeider-oppsatt har vist markant forbedring over en enkeltagent på interne researchevalueringer, i hovedsak fordi arkitekturen tillater langt mer beregning per spørsmål. **Fallgrube:** uten `scope_out` konvergerer arbeiderne mot de samme lett tilgjengelige kildene, og bredden blir illusorisk.

11.2 Storskala kodemigrering

Topologi: pipeline per modul (analyse → transformasjon → test → gjennomgang), med moduler kjørt parallelt. **Hvorfor det virker:** migreringer har maskinell fasit — testene — som gjør validering billig og gjør at feil fanges lokalt i stedet for å forplante seg. Parallelliteten ligger på modulnivå, ikke inne i pipelinen. **Gevinsten kommer fra:** at hver modul får sin egen rene kontekst, slik at et 400-filers prosjekt aldri må ses samlet. **Fallgrube:** delte typer og grensesnitt på tvers av moduler. Disse må migreres først, av ett steg, før modulene slippes løs — ellers får du n innbyrdes inkompatible varianter av samme endring.

11.3 Sikkerhetsvarsling og triage (SOC)

Topologi: orkestrator–arbeider med strengt separerte tilganger. En arbeider har kun lesetilgang til loggkilder, en annen kun til trusseletterretning, en tredje kun til CMDB/eierskapsdata. **Hvorfor det virker:** tilgangsseparasjonen er selve poenget — den arbeideren som leser potensielt fiendtlig innhold, har ingen rettigheter å misbruke. Sammenstillingen skjer hos orkestratoren, som aldri ser rådata. **Fallgrube:** alarmvolumet gjør kostnaden dominerende. Uten et billig førstetrinns filter — regelbasert eller en liten modell — brukes 15× tokenkostnad på varsler som en regex kunne avvist.

11.4 Leverandør- og forsyningskjedeanalyse

Topologi: hypotesedekomponering med eksplisitt uenighetsrapportering, og A2A mot eksterne agenter der motparten selv er en organisasjon med egen agent. **Hvorfor det virker:** spørsmålene er faktaspørsmål med sprikende kilder, og systemets verdi ligger like mye i å rapportere *hvor* kildene spriker som i å konkludere. Signerte Agent Cards gjør det mulig å hente svar fra motparten selv uten å stole blindt på dem. **Fallgrube:** å la modellen harmonisere motstridende kilder til én ryddig konklusjon. Konfliktregelen fra kapittel 8.4 må håndheves i kode, ikke overlates til syntesen.

11.5 Medieproduksjon og transkodingsstyring

Topologi: tavle med én skriver per ressurs, over en tradisjonell jobbkø. **Hvorfor det virker:** agentlaget gjør ikke arbeidet — enkoderen gjør det — men tar *beslutningene*: hvilken bitrate-stige innholdet krever, når en kjøring skal gjøres om, hvordan avvik i kvalitetsmåling skal tolkes. Arbeidsdelingen er god fordi den dyre beregningen er deterministisk og agentlaget billig i sammenligning. **Fallgrube:** reenkoding uten kostnadstak. Én feiltolket VMAF-måling kan utløse tusenvis av jobber; taket hører hjemme i jobbkøen, ikke i prompten.

TABELL 11.1 — OPPSUMMERING: HVOR GEVINSTEN KOMMER FRA

Use case	Topologi	Primær gevinstkilde	Kritisk forutsetning
Dyp research	Orkestrator–arbeider	Mer beregning per spørsmål; parallell kildelesing	Ikke-overlappende mandat
Kodemigrering	Pipeline × moduler	Ren kontekst per modul; maskinell fasit	Delte grensesnitt migreres først
SOC-triage	Orkestrator–arbeider	Tilgangsseparasjon mot fiendtlig inndata	Billig førstetrinns filter
Leverandøranalyse	Hypotese + A2A	Eksplisitt uenighet framfor falsk konsensus	Konfliktregel i kode
Medieproduksjon	Tavle over jobbkø	Beslutninger i agentlaget, arbeid i enkoderen	Kostnadstak i jobbkøen

11.6 Tre bruksområder der multi-agent konsekvent taper

Negative resultater er like nyttige, og disse tre går igjen. **(i) Interaktiv assistanse med sekundkrav.** Koordinering koster veggklokkeid, og brukeren merker det umiddelbart; her vinner én agent med god verktøytilgang hver gang. **(ii) Oppgaver med ett hardt riktig svar og lav tvetydighet** — en SQL-spørring, en enhetskonvertering, en regelbasert klassifisering. Parallellitet legger ikke til informasjon, den legger bare til varians. **(iii) Arbeid der hele materialet må vurderes samtidig**, som å bedømme konsistens i et langt dokument. Kontekstisolasjon er da et rent tap: hver arbeider ser en del og ingen ser helheten, og syntesen kan ikke gjenskape det som gikk tapt.

Fellesnevneren: multi-agent legger til *beregning* og *separasjon* — mangler oppgaven ingen av delene, er arkitekturen kostnad uten motytelse.

Del III · Kapittel 12

Fundamentet: rate, distorsjon og måling

Alle kodekvalg er posisjoner i det samme tredimensjonale rommet: bitrate, kvalitet og beregningskompleksitet. Man kan forbedre to på bekostning av den tredje — aldri alle tre samtidig.

12.1 Rate–distorsjonsoptimalisering

En moderne enkoder tar tusenvis av beslutninger per bilde: blokkstørrelse, prediksjonsmodus, bevegelsesvektor, transformtype. Hver beslutning veies med den samme kostnadsfunksjonen $J = D + \lambda R$, der D er distorsjon (typisk kvadratavvik), R er bitforbruket beslutningen medfører, og λ styrer avveiningen. Denne ene ligningen forklarer det meste av kodeksatferd i praksis: en høyere λ gir en enkoder som systematisk velger billigere, mer unøyaktige representasjoner, og *preset*-innstillingen i en enkoder handler i hovedsak om hvor mange alternativer som faktisk prøves før valget tas.

Konsekvensen er verdt å internalisere: **en kodek er et mulighetsrom, ikke en ytelse**. Nesten alle overraskende sammenligninger — «AV1 var dårligere enn H.264 hos oss» — skyldes en umoden enkoder på rask innstilling målt mot en moden enkoder på treg. Formatet setter taket; enkoder og innstilling bestemmer hvor nær taket man kommer.

12.2 BD-rate: den eneste sammenligningen som betyr noe

Bjøntegaard-Delta-rate (BD-rate) uttrykker den gjennomsnittlige bitratebesparelsen ved lik kvalitet mellom to enkodere, målt over et sett med driftspunkter. Et tall som «–30 % BD-rate mot AV1» betyr: samme opplevde kvalitet med 30 % færre bit. Det er den riktige metrikken fordi den integrerer over hele kurven i stedet for å plukke ett punkt.

TRE MÅTER BD-RATE-TALL VILLEDER

(1) **Metrikkavhengighet.** BD-rate beregnet på PSNR og på VMAF gir ulike tall for samme par — ofte 5–10 prosentpoeng. (2) **Innholdsavhengighet.** Skjerminnhold og HDR viser konsekvent større gevinster enn kamerafanget SDR. (3) **Innstillingsasymmetri.** Sammenligning er bare meningsfull ved sammenlignbart beregningsbudsjett — ellers måler man innstillingen, ikke formatet.

12.3 Kvalitetsmetrikker

TABELL 12.1 — METRIKKER, HVA DE FAKTISK MÅLER, OG HVOR DE SVIKTER

Metrikk	Måler	Styrke	Svakhet du må kjenne
PSNR	Logaritmisk kvadratavvik per piksel	Billig, deterministisk, universelt implementert	Korrelerer svakt med oppfattet kvalitet; straffer nyttig kornstruktur og belønner utglating
SSIM / MS-SSIM	Strukturell likhet i luminans, kontrast og struktur	Bedre samsvar med persepsjon enn PSNR, fortsatt billig	Ufølsom for enkelte tidsmessige artefakter; metter ved høy kvalitet
VMAF	Lært kombinasjon av flere trekk, kalibrert mot menneskelige vurderinger	Best korrelasjon i strømmescenarier; skala 0–100 er tolkbar	Trent på et bestemt innholds- og visningsscenario; kan optimaliseres mot direkte
XPSNR	Vektet PSNR med persepsjonsmodell for høyoppløst innhold	Godt egnet der VMAFs treningsdomene ikke passer, f.eks. UHD/HDR	Mindre utbredt; færre referansepunkter i litteraturen
MOS / DSIS	Faktiske menneskelige vurderinger under kontrollerte forhold	Fasiten alle andre metrikker kalibreres mot	Dyrt og tregt; ikke praktisk i en kontinuerlig leveransekjede

Anbefalt praksis: **optimaliser mot VMAF eller XPSNR, verifiser med MOS på et lite fast utvalg, og oppgi alltid hvilken metrikk et tall stammer fra**. Metrikken i optimaliseringssløyfen blir over tid den systemet er best på — ikke den brukeren opplever.

12.4 Kompleksitetsasymmetrien

Enkoding og dekoding er ikke symmetriske, og forholdet er den viktigste driftsvariabelen i en mediekjede: enkoding skjer én gang per aktivum, dekoding én gang per avspilling — på maskinvare utenfor din kontroll. **Første regel:** skal innholdet ses mange ganger, lønner nesten vilkårlig dyr enkoding seg. **Andre regel:** et format uten maskinvaredekoding hos målgruppen påfører brukeren batteriforbruk som ikke vises i dine egne målinger — regelen som avgjør AV2-tidslinjen i kapittel 14.

TABELL 12.2 — TYPISK RELATIV KOSTNAD PER OPERASJON (H.264-PROGRAMVARE = 1)

Operasjon	H.264	AV1	AV2	Praktisk konsekvens
Enkoding, rask innstilling	1×	3–6×	10–25×	Bestemmer om sanntidsenkoding er mulig uten dedikert maskinvare
Enkoding, kvalitetsinnstilling	10×	60–150×	300+×	Akseptabelt for katalog, uakseptabelt for direktesending
Dekoding, programvare	1×	2–3×	4–6×	Merkes som batteriforbruk og varme på mobil
Dekoding, maskinvare	~0	~0	n/a i dag	Eneste kolonne som betyr noe i massedistribusjon — og den som styrer adopsjonstidslinjen

Videokodeker: full sammenligning

Åtte formater som er relevante i 2026, med kompresjonsevne, lisensregime, maskinvarestøtte og den bruksnisjen hvert av dem faktisk vinner.

TABELL 13.1 — VIDEOKODEKER PER AUGUST 2026. BD-RATE ER OMTRENTLIG, MOT H.264 SOM REFERANSE, MÅLT PÅ PERSEPSJONSMETRIKK VED SAMMENLIGNBART ENKODERBUDSJETT

Format	År	BD-rate	Lisens	Maskinvare-dekodning	Vinner på	Merknad
H.264 / AVC	2003	0 %	Patentpool	Universell	Kompatibilitetsgulvet	Fortsatt eneste formatet som dekodes av praktisk talt alt. Ikke fjern det
HEVC / H.265	2013	−35 %	Tre pooler	Bred	4K/HDR i kringkasting og Apple-økosystemet	Fragmentert lisensiering bremsset webadopsjon permanent
VP9	2013	−33 %	Royaltyfri	Bred	Nettvideo før AV1	I praksis i vedlikeholdsmodus; AV1 erstatter den
AV1	2018	−50 %	Royaltyfri (AOMedia)	Bred fra 2023	Standardvalget for ny strømmedistribusjon	SVT-AV1 gjorde enkoding praktisk. Moden i hele kjeden
VVC / H.266	2020	−55 %	Patentpool	Svært begrenset	Kringkasting og enkelte markeder (særlig Brasil/Japan)	Teknisk sterk, men støtten er stort sett fraværende seks år etter
LCEVC	2020	varierer	Lisensiert	Programvare	Forbedringslag over eksisterende dekodere	Nyttig når man ikke kan bytte basisformat, men vil ha gevinst nå
APV	2024	n/a	Royaltyfri	Voksende	Opptak og produksjon (intra-only, lett redigerbart)	Konkurrent til ProRes/DNxHR, ikke til distribusjonskodeker
AV2	2026	−65 %	Royaltyfri (AOMedia)	Ingen før 2027–28	Framtidig 4K/8K/VR-distribusjon	Spesifikasjon 1.0.0 utgitt 28. mai 2026; ~30 % bedre enn AV1

13.1 Hvordan tabellen skal leses

To kolonner dominerer beslutningen, og de er ikke de man forventer. **Maskinvarekolonnen slår BD-rate-kolonnen** i praktisk talt alle distribusjonsscenarier: et format som er 15 % mer effektivt, men må dekodes i programvare, koster brukeren batteritid og gir deg dårligere avspillingskvalitet på svak maskinvare — en netto forverring selv om båndbredden går ned. **Lisenskolonnen avgjør økosystemet:** HEVCs fragmenterte pool-situasjon er hovedgrunnen til at formatet aldri vant weben, til tross for at det var teknisk konkurransedyktig i 2013. VVC gjentar mønsteret nå.

13.2 Valgmatrise etter bruksområde

TABELL 13.2 — ANBEFALT FORMATVALG PER SCENARIO, AUGUST 2026

Scenario	Primærvalg	Fallback	Begrunnelse
VOD-strømming, bred målgruppe	AV1	H.264 (kompatibilitet), HEVC (Apple-TV-apper)	Royaltyfritt, bred maskinvarestøtte siden 2023, best modenhet i verktøykjeden
Direktesending, lav latens	H.264	AV1 der enkoderkapasiteten finnes	Enkodingstid dominerer; AV1 i sanntid krever fortsatt betydelig maskinvare
Sanntidskommunikasjon (WebRTC)	AV1 eller VP9	H.264	SVC-støtte og royaltyfrihet; forhandles ned per klient
Arkiv og mezzanin	APV eller ProRes	FFV1 (tapsfritt)	Intra-only gir redigerbarhet; distribusjonskodeker er feil verktøy her
Kringkasting (DVB/ATSC 3.0)	HEVC	VVC i utvalgte markeder	Mottakerparken er gitt; formatvalget er en markedsbeslutning, ikke en teknisk
8K, VR og volumetrisk	AV1 i dag	Planlegg for AV2 fra 2028	Der AV2s gevinst er størst, men maskinvaren mangler ennå helt

DEN DYRESTE FEILEN I FORMATVALG

Å bytte format uten å måle på *eget* materiale. Publiserte BD-rate-tall er gjennomsnitt over testsett som sjelden ligner din katalog; animasjon, skjermopptak, sport og film oppfører seg radikalt forskjellig, og gevinsten kan avvike med titalls prosentpoeng i begge retninger. Kjør 20–50 representative klipp gjennom begge kandidatene ved sammenlignbart beregningsbudsjett først — en dags arbeid som ofte endrer konklusjonen.

13.3 Overgangsstrategi: flere formater samtidig er normalen

Spørsmålet er sjelden «hvilket format», men «hvilken kombinasjon, og hvordan velges den per klient». Løsningen er å publisere flere varianter og la forhandlingen skje i manifestet (DASH/HLS), slik at klienten velger det beste formatet den faktisk kan maskinvaredekodet. Oppskrift: H.264 som universelt fallback på få trinn, full stige i AV1, og AV2 som tredje variant først når maskinvarestøtten passerer en forhåndsdefinert terskel.

Del III · Kapittel 14

AV2, enkoderøkosystemet og bitrate-stigen

AV2 er årets viktigste codec-hendelse og samtidig den som skal påvirke driften minst i 2026. Dette kapitlet skiller de to tidslinjene og gir den praktiske oppskriften for det som faktisk skal gjøres nå.

14.1 AV2: hva som ble levert, og hva som gjenstår

AOMedia publiserte AV2-spesifikasjonen v1.0.0 **28. mai 2026**. Formatet er royaltyfritt på samme grunnlag som AV1, og gir omtrent **30 % bedre kompresjon enn AV1** ved samme kvalitet — med rapporterte utslag opp mot 38–40 % på skjerminnhold og HDR, altså der AV1 var svakest. Det plasserer AV2 på linje med eller foran VVC i effektivitet, men uten lisenskostnaden, og det er den kombinasjonen som gjør formatet strategisk viktig.

Det som gjenstår er avgjørende: **maskinvaredekodere ventes først i 2027–2028**, og AV2 har vesentlig høyere dekodingskompleksitet enn AV1. Fram til maskinvaren finnes i målgruppens utstyr, betyr «bedre kompresjon» i praksis lavere båndbreddekostnad for deg og høyere batteriforbruk for brukeren — en byttehandel som er negativ i nesten alle distribusjonsscenarioer i dag.

ANBEFALING: TO SPOR, IKKE ETT

Driftssporet (nå): AV1 er standardvalget for ny distribusjon; H.264 forblir kompatibilitetsgulvet. Ingen AV2 i produksjon i 2026.

Beredskapssporet (nå): sørg for at kjeden er *formatnøytral* — at bitrate-stiger, manifeste, lagringsnavn og kvalitetsmåling ikke har «av1» hardkodet noe sted. Da er AV2-innføring i 2028 en konfigurasjonsendring i stedet for et prosjekt. Dette er hele arbeidet som bør gjøres på AV2 i år, og det er billig.

14.2 Enkoderøkosystemet

TABELL 14.1 — PRODUKSJONSRELEVANTE ENKODERE PER AUGUST 2026

Enkoder	Format	Posisjon	Praktisk råd
SVT-AV1	AV1	De facto produksjonsenkoder; skalerer godt over mange kjerner	Standardvalget. Bruk <code>--preset 4–8</code> for katalog, 10–12 for nær-sanntid
libaom-av1	AV1	Referanseenkoder; høyest kvalitet ved ekstremt lang kjøretid	Bruk til referansemålinger og BD-rate-arbeid, ikke til produksjon
rav1e	AV1	Rust-implementasjon med fokus på hastighet og sikkerhet	Aktuell der minnesikkerhet i enkoderen er et krav
x264 / x265	H.264 / HEVC	Modne, ekstremt godt innstilte	Fortsatt referansen for hva en moden enkoder kan gjøre
VVenC / VVdeC	VVC	Åpne implementasjoner av Fraunhofer	Relevant kun der markedet krever VVC
Maskinvare (NVENC, QSV, AMF)	flere	Sanntid i stor skala til lavere kvalitet per bit	Riktig for direktesending; feil for katalog der bitene betales mange ganger

14.3 Bitrate-stigen: per-tittel slår faste stiger

En fast stige (f.eks. 360p @ 800 kbps, 720p @ 2500 kbps ...) er feil for begge ytterpunkter av en katalog: enkelt animasjonsinnhold får flere bit enn det trenger, komplekst sportsinnhold for få. En **per-tittel-stige** måler faktisk kompleksitet og plasserer trinnene der kvalitetskurven bøyer av. Gevinsten er typisk 15–30 % båndbredde ved uendret kvalitet — sammenlignbart med et helt formathopp, til en brøkdel av risikoen.

```
# 1) Mål kompleksitet: kort CRF-sveip på et representativt utvalg av tittelen
for crf in 20 26 32 38; do ffmpeg -i inn.mov -c:v libsvtav1 -crf $crf -preset 6 -g 240 \
    -an -f null - 2>> sveip.log ; done

# 2) Enkod ett trinn, og mål VMAF mot kilden (skalere referansen, ikke omvendt)
ffmpeg -i inn.mov -c:v libsvtav1 -crf 30 -preset 6 -g 240 -pix_fmt yuv420p10le \
    -c:a libopus -b:a 96k -movflags +faststart ut_1080p.mp4
ffmpeg -i ut_1080p.mp4 -i inn.mov -lavfi "[0:v]scale=1920:1080[d];[1:v]scale=1920:1080[r];\
[d][r]libvmaf=n_threads=8" -f null -

# 3) Velg trinn der VMAF-gevinsten per ekstra Mbit/s faller under ~2 VMAF/Mbps
```

14.4 Tre driftsråd som gir mest effekt

- **Bruk 10-bits koding også for 8-bits kilder.** Med AV1 gir `yuv420p10le` typisk bedre rate–distorsjon enn 8 bit selv når kilden er 8 bit, fordi intern presisjon reduserer akkumulert kvantiseringsstøy. Kostnaden er marginal.
- **Fest nøkkelbildeintervallet til segmentlengden.** Et GOP som ikke er et helt multiplum av segmentlengden, tvinger fram ekstra nøkkelbilder ved segmentgrensene og spiser mellom 5 og 15 % av budsjettet uten kvalitetsgevinst.
- **Mål alltid mot kilden, aldri mot forrige generasjon.** Kvalitetsmåling mot en tidligere transkodning skjuler akkumulert tap, og gir en katalog som degraderes systematisk over flere migreringer uten at noen måling viser det.

Del III · Kapittel 15

Lydkodeker

Lydsiden er mer avklart enn videosiden: Opus vinner sanntid og det åpne nettet, AAC-familien vinner distribusjon, og valget mellom dem avgjøres av dekodeparken — ikke av kvalitet.

TABELL 15.1 — LYDKODEKER PER AUGUST 2026

Kodek	Nyttig bitrate	Latens	Lisens	Vinner på	Merknad
Opus	6–510 kbps	5–66 ms	Royaltyfri	Sanntid, WebRTC, tale og musikk i ett format	Bytter sømløst mellom SILK (tale) og CELT (musikk) etter innhold og bitrate
AAC-LC	96–256 kbps	~50 ms	Patentpool	VOD-distribusjon — det trygge standardvalget	Dekodes av praktisk talt alt; derfor fortsatt førstevalg for kompatibilitet
HE-AAC v1/v2	24–80 kbps	~130 ms	Patentpool	Lav bitrate der AAC-LC svikter	SBR/PS gir god lav-bitrate-ytelse, men merkbar tilleggs-latens
xHE-AAC (USAC)	12–256+ kbps	middels	Patentpool	Adaptiv strømming over hele bitratespennet	Det moderne AAC-valget; leveres som standard i nyere Android
LC3 / LC3plus	16–320 kbps	<10 ms	Lisensiert	Bluetooth LE Audio og profesjonelt lydutstyr	Erstatter SBC; LC3plus tilføyer høyoppløst og svært lav latens
FLAC	~50 % av PCM	n/a	Royaltyfri	Tapsfri arkivering og distribusjon	Bredt støttet, enkel, moden. Standardvalget for tapsfritt
ALAC	~50 % av PCM	n/a	Royaltyfri	Tapsfritt i Apple-økosystemet	Teknisk jevnbyrdig med FLAC; valget er økosystem, ikke ytelse

15.1 Hvorfor Opus fortsatt vinner sanntid

Opus løser et problem ingen andre løser like godt: **ett format som dekker hele spennet fra 6 kbps tale til transparent musikk, med latens ned mot 5 ms, uten lisenskostnad.** Hybridarkitekturen — SILK for tale ved lave bitrater, CELT for musikk og lav latens, med sømløs overgang — betyr at systemet ikke må velge format på forhånd. Det er også obligatorisk i WebRTC, som gjør det til det eneste formatet man kan anta at motparten støtter i nettleserbasert sanntidskommunikasjon. Femten år etter er det fremdeles ingen reell konkurrent i den nisjen.

15.2 Valgguide

TABELL 15.2 — ANBEFALT LYDKODEK PER SCENARIO

Scenario	Valg	Begrunnelse og typisk innstilling
Nettmøter, tale, WebRTC	Opus	24–32 kbps mono for tale; slå på DTX og FEC ved pakketap
Musikkstrømming	xHE-AAC eller Opus	Opus ~128 kbps stereo er transparent for de fleste; xHE-AAC der dekodeparken krever AAC
VOD-lyd til video	AAC-LC	128–192 kbps stereo. Kompatibilitet er verdt mer enn de siste prosentene effektivitet
Podkast / tale-VOD	Opus 48–64 kbps	Halverer båndbredden mot AAC ved samme oppfattede talekvalitet
Trådløse ørepropper	LC3	Gitt av Bluetooth LE Audio; ikke et fritt valg i praksis
Arkiv / master	FLAC	Tapsfritt, åpent, og lesbart om tjue år. Ikke arkiver i et tapsbasert format
Tale inn i ASR/LLM	Opus eller PCM	16 kHz mono holder for ASR; unngå kaskadetranskoding, som skader gjenkjenningssraten

DEN VANLIGSTE LYDFEILEN I EN AGENT- OG MEDIEKJEDE

Kaskadetranskoding. Lyd som går Opus → AAC → Opus → PCM gjennom flere tjenesteteg, akkumulerer tap ved hvert steg, og resultatet er hørbart dårligere enn noen av leddene tilsier. I ASR- og tale-til-tale-kjeder gir dette målbart fall i gjenkjenningssrate. Løsningen er arkitektonisk, ikke innstillingsbasert: **bestem ett internt arbeidsformat** — normalt PCM eller Opus — og transkod kun én gang, ytterst i kjeden, ved levering.

15.3 Loudness og kanaloppsett: der de fleste faktiske klagen kommer fra

Kodekvalget er sjelden årsaken til at brukere opplever dårlig lyd. To andre forhold står for de fleste klagen, og begge er billige å løse. **Loudness-normalisering:** uten en felles målverdi varierer opplevd volum mellom klipp, og brukeren justerer volumet konstant. Normaliser til en fast integrert loudness (EBU R 128 med -23 LUFS for kringkasting, -16 til -14 LUFS for nettdistribusjon), og lagre målverdien som metadata slik at avspilleren kan justere uten å re-encode. **Nedmiksing:** feil nedmikset flerkanaalslyd gir dialog som drukner i effektlyden. Test 5.1 → stereo eksplisitt; standardverdiene er sjelden riktige for dialogtungt materiale.

```
# Toppass loudness-normalisering (måling først, så korreksjon – ett enkeltpass er upresist)
ffmpeg -i inn.wav -af loudnorm=I=-16:TP=-1.5:LRA=11:print_format=json -f null - # pass 1
ffmpeg -i inn.wav -af loudnorm=I=-16:TP=-1.5:LRA=11:measured_I=-21.3:measured_TP=-3.1:\
measured_LRA=7.4:measured_thresh=-31.8:linear=true -c:a libopus -b:a 96k ut.opus # pass 2
```

Del III · Kapittel 16

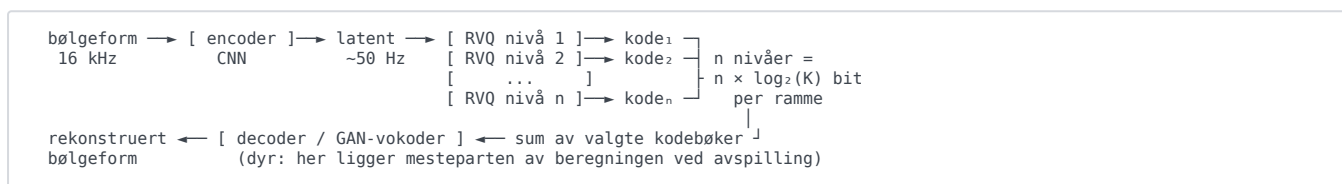
Nevrale codecs

Nevrale lydkodeker oppnår forbløffende kvalitet ved bitrater der klassiske kodeker faller sammen. De har likevel ett dominerende bruksområde i produksjon i dag — og det er ikke båndbreddesparing.

16.1 Arkitekturen: encoder, RVQ, decoder

Praktisk talt alle moderne nevrale lydkodeker deler samme oppbygning. En **konvolusjonsenkoder** komprimerer bølgeformen til en latent med lav rammerate (typisk 12,5–75 rammer/s mot 16–48 kHz samples). Latenten kvantiseres av en **residual vektorkvantiserer (RVQ)** — en kaskade av kodebøker der hvert nivå koder feilen forrige nivå etterlot — og en **dekoder**, ofte en GAN-trent vokoder, rekonstruerer bølgeformen.

RVQ-strukturen gir en egenskap klassiske kodeker mangler: **bitraten er en kjøretidsparameter**. Fire av åtte kodebøker gir halv bitrate uten modellbytte, fordi hvert nivå bare forfiner — naturlig skalerbarhet ved varierende nettverk.



Figur 16.1 — Encoder–RVQ–decoder. Antall aktive RVQ-nivåer settes ved kjøretid og gir kontinuerlig bitrateskalering.

16.2 Sammenligning

TABELL 16.1 — REPRESENTATIVE NEVRALE LYDKODEKER

Modell	Bitrate	Rammerate	Designmål	Praktisk vurdering
SoundStream / Lyra	3–6 kbps	50 Hz	Tale over svært dårlige nett	Første som viste at nevralkoding var praktisk; i dag først og fremst historisk viktig
EnCodec	~6 kbps	75 Hz	Generell lyd, høy kvalitet, RVQ-basert	Mye brukt som tokeniser i forskning; god referanseimplementasjon
Mimi (Moshi)	~1,1 kbps	12,5 Hz	Sanntids samtale, ikke offline rekonstruksjon	Bygget for tale-til-tale-modeller; den lave rammeraten er poenget, ikke bitraten
SNAC	~1–3 kbps	flerskala	Hierarkisk tidsoppløsning i RVQ-nivåene	Godt egnet der en språkmodell skal generere token på ulike tidsskalaer
DAC	~8 kbps	86 Hz	Høy generell lyd kvalitet, bredt innholdsspenn	Blant de sterkeste på musikk; tyngre å kjøre enn taleorienterte modeller

16.3 Det egentlige bruksområdet: taletokens til språkmodeller

Den viktigste anvendelsen av nevrale codecs i 2026 er ikke å spare båndbredde. Det er å gjøre lyd om til **diskrete token en språkmodell kan behandle direkte**. En kodek med 12,5 rammer per sekund og noen få RVQ-nivåer produserer en tokenstrøm i samme størrelsesorden som tekst, og gjør det mulig å modellere tale ende-til-ende — uten å gå veien om tale-til-tekst-til-tale. Det er denne egenskapen, ikke kompresjonsraten, som gjør Mimi og lignende modeller interessante: de er designet for strømmende samtale, med en rammerate valgt for å passe en transformer, ikke et øre.

ANBEFALING: NEVRAL CODEC ER ET FORSKNINGSVALG, IKKE ET DRIFTSVALG

For ordinær sanntidskommunikasjon er **Opus fortsatt riktig standardvalg**. Nevrale codecs har tre kostnader som sjelden nevnes sammen med kvalitetstallene: **(i) beregning ved dekodning** — vokoderen kjører på hver klient, ofte uten maskinvarestøtte; **(ii) innholdsavhengig degradering** — modeller trent på tale feiler uforutsigbart på musikk eller flere samtidige stemmer, og feilen høres ut som feil *innhold*, ikke som artefakter; og **(iii) modellstyring** — koder og dekker må matche i versjon, noe som gjør oppgraderinger til et distribusjonsproblem. Velg nevral der nettverket er den harde begrensningen og beregningskraften finnes, eller der du trenger taletoken. Ellers ikke.

16.4 Nevral videokoding: hvorfor bildet er et annet

Hvis nevrale kodeker slår klassiske på lyd ved lave bitrater, hvorfor ikke på video? Fordi beregningsasymmetrien går motsatt vei: en lydvokoder kan kjøre i sanntid på en telefon, mens en nevralkodekoder må produsere millioner av piksler per sekund uten den dedikerte maskinvaren klassiske formater har hatt i tju år. Forskningsresultatene er reelle, men de forutsetter GPU-dekodning hos mottakeren — noe massedistribusjon ikke kan forutsette.

Der nevrale metoder allerede har praktisk verdi, er **som komponenter i en ellers klassisk pipeline**: lært oppskalering på klientsiden, lært støyfjerning før enkoding, og lært valg av enkoderparametere per scene. Alle tre gir gevinst uten at mottakeren må bytte dekker — og det er nettopp den egenskapen som avgjør om noe kan tas i bruk i år eller om tre.

Del III · Kapittel 17

Data-, tekst- og serialiseringskompresjon

Den delen av kompresjonsfeltet som berører flest systemer, og der et enkelt, velbegrunnet standardvalg gir mest igjen for innsatsen.

17.1 Generelle kompresjonsalgoritmer

TABELL 17.1 — STØRRELSESORDENER PÅ MODERNE MASKINVARE. RATIO MOT UKOMPRIMERT TEKST; HASTIGHETER ER PER KJERNE

Algoritme	Ratio	Komprimering	Dekomprimering	Riktig bruksområde
LZ4	~2,1×	~500 MB/s	~2500 MB/s	Der kompresjon må være «gratis»: RAM-cacher, hot path, midlertidige filer
Snappy	~2,1×	~400 MB/s	~1500 MB/s	Eldre Hadoop/Kafka-oppsett; ingen grunn til å velge nytt i dag
zstd -1	~2,9×	~450 MB/s	~1300 MB/s	Erstatter LZ4 i de fleste tilfeller: bedre ratio, sammenlignbar fart
zstd -3 (standard)	~3,2×	~250 MB/s	~1200 MB/s	Standardvalget. Slår gzip på både ratio og hastighet, i begge retninger
zstd -19	~4,0×	~5 MB/s	~1000 MB/s	Skriv-én-gang-les-mange: artefakter, images, kaldt arkiv
gzip -6	~2,8×	~60 MB/s	~350 MB/s	Kun der mottakeren ikke kan noe annet. Ellers utdatert
brotili -11	~4,3×	~1 MB/s	~450 MB/s	Statiske webressurser komprimert på forhånd; innebygd webordbok gir ekstra gevinst på HTML/JS
xz / LZMA -9	~4,5×	~2 MB/s	~80 MB/s	Maksimal ratio der dekomprimeringstid ikke betyr noe. Merk den lave leseraten

17.2 Ordbøker: den mest oversette optimaliseringen

Når man komprimerer mange små, likeartede objekter — JSON-meldinger, loggrader, API-responser — er problemet at hvert objekt er for lite til at algoritmen rekker å lære mønstrene. En **forhåndstrent ordbok** gir kompressoren en felles kontekst som ikke må sendes med. På JSON-meldinger under noen kilobyte er gevinsten ofte 2–5× bedre ratio, uten tap i hastighet.

```
# Tren en ordbok på et representativt utvalg (noen tusen meldinger holder)
zstd --train prover/*.json -o meldinger.dict --maxdict=112640

# Bruk den i begge retninger – koder og dekkerer må ha samme ordbok, versjonér den som en artefakt
zstd -D meldinger.dict -3 melding.json -o melding.json.zst
zstd -d -D meldinger.dict melding.json.zst
```

Ordboken må versjoneres og distribueres som en artefakt på lik linje med kode. En ordbok som ikke matcher mellom koder og dekkerer, gir ikke dårligere ratio — den gir feil.

17.3 Kolonneformater og serialisering

For analytiske data er valget av *format* mer virkningsfullt enn valget av algoritme. Kolonneformater som Parquet og ORC lagrer verdier av samme type sammen, hvilket gjør spesialiserte teknikker mulige før den generelle kompresjonen i det hele tatt starter: **ordbokkoding** for gjentatte strenger, **run-length** for sorterte kolonner, **delta** for tidsstempler og **bit-packing** for tallkolonner med lite spenn. Effekten er større enn noen algoritmeoppgradering kan gi — det er vanlig å se 5–20× mot rad-basert lagring. Praktisk råd: **sortér på kolonnen med lavest kardinalitet før skriving**, og slå på statistikk per radgruppe, slik at leseren kan hoppe over blokker uten å dekomprimere dem.

For meldinger over nett gjelder en beslektet regel: et selvbeskrivende tekstformat (JSON) koster både båndbredde og parsetid. Protobuf, Avro og CBOR fjerner feltnavnene fra nyttelasten og er typisk 3–10× mindre *før* kompresjon. Kombinasjonen som gir mest i praksis er **skjemabasert serialisering + zstd med ordbok** — men bare der volumet rettferdiggjør tapet i lesbarhet under feilsøking.

ANBEFALING I ÉN SETNING

Standardisér på zstd, med nivå per arbeidsmengde (–1 varmt, –3 generelt, –19 kaldt) og ordbøker der objektene er små og likeartede. Behold brotili for forhåndskomprimerte statiske webressurser, og gzip kun der en mottaker krever det. Ett bibliotek som dekker hele spennet fra LZ4-hastighet til nær-xz-ratio, er verdt mer i drift enn de siste prosentene fra et spesialisert valg per system.

17.4 Kontekstkomprimering: samme problem, nytt lag

Agentsystemer har gjenoppdaget kompresjonsproblemet i ny form. Kontekstvinduet er en kanal med fast kapasitet, og spørsmålet «hva kan fjernes uten at mottakeren merker det» er det samme en tapsbasert kodek stiller. De fire etablerte teknikkene har hver sin klassiske analogi: **oppsummering ved terskel** = tapsbasert koding; **henting ved behov** (RAG) = referansebasert koding, der en peker sendes i stedet for innholdet; **strukturerte sammendrag med faste felt** = skjemabasert serialisering; og **prompt-caching** = ordbokgjennbruk.

Lærdommen overføres direkte fra mediefeltet: **komprimér mot kilden, ikke mot forrige generasjon**. Et sammendrag av et sammendrag akkumulerer tap som kaskadetranskoding — usynlig i hvert steg, tydelig i resultatet.

Arkitektur, overflater og kontekstøkonomi

Claude Code er en agentkjøring med filsystem-, kommando- og verktøytilgang, som eksponeres gjennom flere overflater mot samme motor. Konfigurasjon, instruksjoner og MCP-servere følger med på tvers.

18.1 Overflatene og hva de faktisk brukes til

TABELL 18.1 — OVERFLATER OG DERES REELLE BRUKSMØNSTER

Overflate	Egner seg til	Merknad fra praksis
Terminal (CLI)	Hovedarbeidsflaten; full funksjonalitet, skripting og piping	Følger Unix-filosofien: <code>claude -p</code> gjør den til en komponent i en pipeline, ikke bare et chatvindu
VS Code / JetBrains	Inline differ, @-referanser, plangjennomgang i editoren	Best der gjennomgang av endringer er hovedaktiviteten
Desktop-app	Flere parallelle sesjoner, visuell diff-gjennomgang, planlagte oppgaver	Naturlig sted å overvåke flere agenter samtidig
Web og mobil	Langvarige oppgaver uten lokalt oppsett; arbeid på repo du ikke har lokalt	Kjører i isolert, kortlivet miljø — alt som skal beholdes må committes og pushes
CI/CD	Automatisk PR-gjennomgang, issue-triage, planlagte revisjoner	GitHub Actions og GitLab CI/CD; her er determinisme viktigere enn interaktivitet
Chat (Slack)	Feilrapport inn, pull request ut	Lav terskel for ikke-utviklere; krever stramme tillatelser

Det arkitektonisk viktige er at overflatene deler **samme motor og samme konfigurasjonshierarki**: en `CLAUDE.md`, en skill eller en MCP-serverdefinisjon virker uendret i CI og i editoren. Derfor: **invester i primitivene, ikke i overflatespesifikke tilpasninger**.

18.2 Sesjonsmodellen og kontekstøkonomien

Kontekstvinduet er den knappeste ressursen i en agentsesjon, og de fleste kvalitetsproblemer som ser ut som modellproblemer, er kontekstproblemer. Tre mekanismer forvalter det:

- **Delegering til subagent** — mest effektivt: arbeid som produserer materiale du ikke skal se igjen (søk, loggjakt, filgjennomgang) gjøres i egen kontekst, og bare konklusjonen returneres. Dette er hovedgrunnen til at subagenter finnes.
- **Komprimering** — samtalen oppsummeres når vinduet fylles. Nyttig, men tapsbasert; jf. kapittel 17.4.
- **Vedvarende instruksjoner** — `CLAUDE.md` og automatisk minne bærer det som skal overleve på tvers av sesjoner. Billigst av alt, fordi det ikke må gjenoppdages.

18.3 Konfigurasjonshierarkiet

1. Managed policy settings — organisasjonsstyrt, kan ikke overstyres av bruker/prosjekt
2. `~/.claude/settings.json` — brukers egne, på tvers av prosjekter
3. `.claude/settings.json` — prosjektets, versjonert i repoet
4. `.claude/settings.local.json` — lokal overstyring, gitignored
5. Plugin-hooks (`hooks/hooks.json`) — følger med installerte plugins
6. Skill-/agent-frontmatter — gjelder kun den skillen eller agenten

Merk: hooks fra ALLE nivåer slås sammen — ingen erstattes. Managed hooks kan ikke slås av.

PRAKTISK KONSEKVENNS FOR TEAM

Det som *må* gjelde alle, hører i managed policy settings; det som gjelder prosjektet, i `.claude/settings.json` (versjonert, gjennomgått i PR); personlige preferanser i `settings.local.json`. Team som legger prosjektreglene i brukerinnstillinger, får oppførsel som avhenger av hvem som kjører — den vanskeligste klassen av «virker på min maskin».

18.4 Tillatelsesmodeller og sikkerhetsgrenser

Fire samvirkende mekanismer dekker ulike trusler — og forskjellen mellom dem avgjør hvilken innstilling som er riktig per miljø:

- **Tillatelsesmodus** styrer hvor mye som utføres uten å spørre. Den skiller *ikke* mellom trygge og skadelige kommandoer.
- **Tillatelsesregler** avgjør hvilke verktøy og kommandomønstre som tillates, spørres om eller nektes. En tillatt kommando med farlige argumenter slipper likevel gjennom.
- **Hooks** er den eneste programmerbare porten: vilkårlig egen logikk før og etter verktøybruk. Dekker det du faktisk har skrevet en kontroll for — ikke mer.
- **Sandkasse** begrenser konsekvensen når noe likevel går galt: filsystem, nettverk, ressurser.

Bruk alle fire, med ulik stramhet per miljø: interaktivt lokalt arbeid tåler en åpnere modus fordi et menneske ser på; autonome kjøring i CI eller på tidsplan trenger strenge regler, blokkerende hooks og reell sandkasse — nettopp fordi ingen ser på.

Utvidelsesprimitivene: hva du bruker når

Seks primitiver med hvert sitt formål. Den vanligste feilen er å bruke ett av dem til alt — typisk å skrive lange instruksjoner i `CLAUDE.md` der en hook eller en skill hadde løst problemet skikkelig.

TABELL 19.1 — DE SEKS PRIMITIVENE: FORMÅL, DETERMINISME OG KONTEKSTKOSTNAD

Primitiv	Løser	Determi- nistisk?	Kontekst- kostnad	Bruk når
CLAUDE.md / minne	Vedvarende kontekst: standarder, arkitektur, kommandoer	Nei	Alltid lastet	Informasjon som gjelder <i>hele tiden</i> og er kort. Hold den stram — alt her betaler token hver sesjon
Skills	Gjenbrukbare arbeidsflyter pakket som instruksjoner + hjelpefiler	Nei	Ved behov	Prosedyrer som brukes av og til: utgivelsesrutine, gjennomgangssjekkliste, rapportmal
Subagenter	Isolert kontekst, egne verktøygrenser, parallelitet	Nei	Egen kontekst	Sidearbeid som ville fylt hovedkonteksten, eller arbeid som skal ha strammere tilganger
Hooks	Deterministisk kjøring ved livssyklushendelser	Ja	Ingen	Alt som må skje uansett: formatering, hemmelighetsskanning, blokkering, revisjonslogg
MCP-servere	Tilgang til eksterne systemer og data	Delvis	Verktøyliste	Integrasjon mot systemer utenfor filsystemet: Jira, database, intern API
Plugins	Versjonert distribusjon av alt det ovenstående	n/a	Arvet	Når mer enn ett team skal bruke samme oppsett, og det må kunne versjoneres og ruller tilbake

19.1 Beslutningsregelen

Én regel avgjør de fleste tvilstilfeller, og den bør stå eksplisitt i teamets retningslinjer:

REGEL

Må det skje, er det en hook. Bør det skje, er det en skill eller en instruksjon. En instruksjon i `CLAUDE.md` er en anmodning modellen normalt følger, men kan velge bort under press fra andre hensyn. En hook er kode som kjører. Sikkerhetskontroller, formatering, hemmelighetsskanning og revisjonslogging hører derfor alltid i hooks — aldri i prompten. Samme prinsipp som «autorisasjon håndheves utenfor modellen» (kap. 4.2), anvendt på verktøynivå.

19.2 Tre typiske feilplasseringer

TABELL 19.2 — VANLIGE FEIL OG RIKTIG PLASSERING

Det man ofte gjør	Hvorfor det svikter	Riktig plassering
«Kjør alltid <code>prettier</code> etter endring» i <code>CLAUDE.md</code>	Følges som regel, men ikke alltid — og avviket oppdages i PR-en, ikke lokalt	<code>PostToolUse</code> -hook på <code>Edit Write</code> som faktisk kjører formateringen
En 600 linjers <code>CLAUDE.md</code> med all tenkelig kontekst	Betales i token hver eneste sesjon, og det viktige drukner i det sjeldne	Kort <code>CLAUDE.md</code> for det som alltid gjelder; resten som skills som lastes ved behov
«Aldri rør produksjonsdatabasen» som instruksjon	En instruksjon er ikke en kontroll; indirekte prompt-injeksjon omgår den	<code>PreToolUse</code> -hook som returnerer <code>permissionDecision: "deny"</code> , kombinert med tillatelsesregler

19.3 En anbefalt startpakke

Denne rekkefølgen gir mest effekt: **(1)** stram `CLAUDE.md`; **(2)** `PostToolUse` -hook som formaterer og linter; **(3)** `PreToolUse` -hook mot destruktive kommandoer og hemmeligheter; **(4)** én subagent for utforskning; **(5)** én MCP-server mot systemet teamet oftest slår opp i.

19.4 En `CLAUDE.md` som faktisk virker

Den vanligste feilen er lengde. En god `CLAUDE.md` svarer på det en ny kollega ville spurt om den første dagen — og ikke mer.

```
## Kommandoer    ← viktigst: sparer agenten for å gjette seg fram
Bygg `pnpm build` · Test `pnpm test` · Lint `pnpm lint --fix` · DB: `docker compose up -d db`
## Arkitektur i tre setninger
Fastify-API mot PostgreSQL. Domenelogikk i `src/domain` er rammeverksfri og importerer
aldri fra `src/http`. All persistens går gjennom repositories i `src/db`.
## Konvensjoner
- Ingen `any` – bruk `unknown`. - Feil: kast `DomainError`, aldri strenger.
- Nye endepunkter krever kontrakttest i `test/contract`.
## Fallgruver vi har gått i    ← høyest verdi per linje i hele fila
- `migrate:dev` sletter lokale data. Bruk `migrate:up` hvis du ikke vil nullstille.
- Testene deler databaseskjema – kjør sekvensielt (`--runInBand`), ikke parallelt.
```

Merk hva som *ikke* står der: ingen sikkerhetsregler (hooks), ingen lange prosedyrer (skills). «Fallgruver vi har gått i» gir mest igjen per linje.

Hooks: det deterministiske kontrollpunktet

Hooks er shell-kommandoer, HTTP-kall, MCP-verktøy eller egne prompter som kjører ved definerte livssyklus hendelser. De er den eneste mekanismen som garanterer at noe skjer.

20.1 Hendelsene som betyr mest

TABELL 20.1 — DE VIKTIGSTE HOOK-HENDELSENE (UTVALG; DET FINNES OVER TJUE)

Hendelse	Utløses	Matcher	Kan blokkere	Typisk bruk
SessionStart	Sesjon starter eller gjenopptas	startup, resume, clear, compact, fork	Nei	Laste inn prosjektkontekst, verifisere at miljøet er klart
UserPromptSubmit	Bruker sender inn en melding	—	Ja	Legge på kontekst, avvise forespørsler som bryter policy
PreToolUse	Før et verktøy kjøres	verktøynavn	Ja	Hovedsikkerhetsporten. Nekte, tillate, omskrive argumenter
PermissionRequest	Et verktøy trenger tillatelse	verktøynavn	Ja	Automatisk godkjenning av kjente trygge operasjoner
PostToolUse	Etter vellykket verktøykjøring	verktøynavn	Ja	Formatering, linting, skanning, maskering av utdata
SubagentStart / Stop	Subagent starter/avslutter	agenttype	Delvis	Budsjettsporing og revisjonslogg per delegering
Stop	Claude avslutter svaret	—	Ja	Blokkere avslutning til tester består; legge til sluttkontekst
SessionEnd	Sesjon avsluttes	clear, resume, logout, ...	Nei	Opprydding, eksport av revisjonsspor

20.2 Kontrakten: exit-koder og JSON

En hook kommuniserer på to måter. **Exit-koden:** 0 = suksess (stdout tolkes som JSON), 2 = blokkerende feil (stderr blir begrunnelsen), annet = ikke-blokkerende feil. **JSON på stdout** er det presise grensesnittet:

```
{
  "continue": true,
  "systemMessage": "Advarsel: bruker utfaset API",
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Skriving mot produksjonsdatabase er ikke tillatt herfra",
    "updatedInput": { "command": "psql $STAGING_URL -c '...' },
    "additionalContext": "Bruk staging-tilkoblingen; produksjon krever endringsordre."
  }
}
```

20.3 Et komplett, produksjonsnært oppsett

```
// .claude/settings.json – versjonert i repoet, gjennomgått i pull request
{
  "hooks": {
    "PreToolUse": [ { "matcher": "Bash", "hooks": [
      { "type": "command", "if": "Bash(rm *)",
        "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/guard-rm.sh", "timeout": 10 },
      { "type": "prompt", "if": "Bash(git push *)",
        "prompt": "Er denne pushen trygg gitt gjeldende branch-policy? $ARGUMENTS" } ] ] },
    "PostToolUse": [ { "matcher": "EditWrite", "hooks": [
      { "type": "command", "command": "pnpm",
        "args": ["exec", "prettier", "-w", "${tool_input.file_path}"] },
      { "type": "mcp_tool", "server": "security", "tool": "scan_file",
        "input": { "path": "${tool_input.file_path}" } } ] ] },
    "SubagentStop": [ { "hooks": [ { "type": "command", "async": true,
      "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/log-delegation.sh" } ] ] }
  ]
}
```

```
#!/usr/bin/env bash
input=$(cat); cmd=$(jq -r '.tool_input.command' <<<"$input")
if [[ "$cmd" =~ rm[:space:]]+([-a-zA-Z]*[rf])[-a-zA-Z]*[:space:]]+ */~|~$HOME/\.|\. ]]; then
  jq -n '{hookSpecificOutput:{hookEventName:"PreToolUse",permissionDecision:"deny",
    permissionDecisionReason:"Rekursiv sletting utenfor prosjektkatalogen er blokkert."}}'
  exit 0
fi
exit 0
```

TRE FEIL SOM GJØR HOOKS UPÅLITELIGE

(1) **Manglende timeout.** En hook som henger, henger sesjonen. Sett alltid en grense. (2) **Blokkering uten begrunnelse.** Nekt alltid med `permissionDecisionReason` — uten den ser agenten bare at noe feilet og prøver ofte en variant av samme handling. (3) **Tung logikk synkront.** Logging og varsling hører til `"async": true`; bare det som må påvirke beslutningen, skal kjøre synkront.

Subagenter, skills og plugins

Tre mekanismer for gjenbruk, med hver sin filstruktur. Skills pakker kunnskap, subagenter pakker isolert arbeid, plugins pakker begge deler for distribusjon.

21.1 Subagenter

En subagent er en markdown-fil med YAML-frontmatter, plassert i `.claude/agents/` (prosjekt) eller `~/.claude/agents/` (bruker). Den får egen kontekst, egen systemprompt, egne verktøygrenser og egne tillatelser. `description`-feltet er det viktigste: det er dette hovedagenten bruker for å avgjøre om oppgaven skal delegeres hit.

```
--- .claude/agents/db-migrasjon.md ---
---
name: db-migrasjon
description: Analyserer og skriver databasemigrasjoner. Brukes når skjemaendringer,
  indekser eller migrasjonsfiler er involvert. Skriver ALDRI mot produksjon.
tools: Read, Grep, Glob, Edit, Write, Bash      # utelatte verktøy er utilgjengelige
model: sonnet                                  # billigere modell for avgrenset arbeid
---
Du skriver databasemigrasjoner for et PostgreSQL-skjema.

Ufravikelige regler:
1. Hver migrasjon skal ha en verifisert `down`-operasjon. Kan den ikke reverseres,
  si fra eksplisitt i stedet for å skrive den.
2. Legg aldri til en NOT NULL-kolonne uten default på en tabell med data.
3. Indeksbygging på store tabeller skal bruke CONCURRENTLY.
4. Returnér alltid: (a) migrasjonsfilen, (b) estimert låsetid, (c) rollback-plan.
```

Verdien ligger i to ting: **kontekstisolasjon** og **tvangsmessig avgrensning** — `tools`-listen er en teknisk grense, ikke en anmodning. Den siste er mest undervurdert: en subagent uten skrivende verktøy *kan* ikke gjøre skade, uansett hva den blir bedt om.

21.2 Skills

En skill er en katalog med en `SKILL.md` og eventuelle hjelpefiler og skript. Den lastes **ved behov** — beskrivelsen ligger tilgjengelig, men innholdet leses først når oppgaven matcher. Det gjør skills til riktig sted for prosedyrer som brukes av og til, og som ville vært sløsing å ha i konteksten hele tiden.

```
.claude/skills/utgivelse/
├── SKILL.md           # frontmatter (name, description) + selve prosedyren
├── sjekkliste.md      # lastes bare hvis SKILL.md henviser til den
└── scripts/lag-changelog.sh

--- SKILL.md ---
---
name: utgivelse
description: Kjør prosjektets utgivelsesprosedyre – versjonsbump, changelog, tagging
  og publisering. Brukes ved "release", "utgivelse", "publiser ny versjon".
---
1. Verifiser at `main` er grønn og at arbeidskatalogen er ren.
2. Bestem semver-nivå ut fra commits siden forrige tag (se sjekkliste.md).
3. Kjør `scripts/lag-changelog.sh` og les gjennom resultatet – ikke publiser ukontrollert.
4. Tag, push tag, og verifiser at publiseringsjobben starter.
```

21.3 Plugins: distribusjon og versjonering

En plugin er en versjonert bunt som kan inneholde skills, subagenter, slash-kommandoer, hooks, output styles og MCP-serverdefinisjoner — alt installerbart som én enhet. Dette er den kanoniske måten å dele oppsett på tvers av team, og den eneste som gir versjonering og tilbakerulling.

TABELL 21.1 — NÅR DU VELGER HVA

Situasjon	Velg	Begrunnelse
Bare mitt prosjekt	<code>.claude/</code> i repoet	Versjoneres med koden, gjennomgåås i PR, ingen ekstra distribusjonsmekanisme
Alle mine prosjekter	<code>~/.claude/</code>	Personlige preferanser. Ikke legg teamregler her
Flere team	Plugin	Versjonering, tilbakerulling og ett sted å rette feil. Signér bunten (kap. 2.3)
Hele organisasjonen, obligatorisk	Managed policy settings	Kan ikke slås av lokalt — riktig for sikkerhetskontroller

ANBEFALING

Start i `.claude/` i repoet. Flytt til plugin først når det tredje teamet ber om samme oppsett — da vet du hva som faktisk er felles, og slipper en abstraksjon bygget på to datapunkter.

Del IV · Kapittel 22

MCP-integrasjon og Agent SDK

To ulike integrasjonsretninger: MCP utvider hva agenten kan gjøre, Agent SDK lar deg bygge din egen agent på samme motor. De brukes ofte sammen.

22.1 Å koble til MCP-servere

Servere defineres i `.mcp.json` i prosjektrøten (versjonert, deles med teamet) eller i brukerens innstillinger. Tre transporttyper dekker alle tilfeller: **stdio** for lokale prosesser, **http** for fjern tjenester, og **sse** som er utfaset til fordel for Streamable HTTP.

```
// .mcp.json – versjonert i repoet
{
  "mcpServers": {
    "postgres": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-postgres",
        "$DB_URL"
      ],
      "env": {
        "DB_URL": "${STAGING_DATABASE_URL}"
      },
      // staging, ikke prod
    },
    "jira": {
      "type": "http",
      "url": "https://mcp.intern.no/jira",
      "headers": {
        "Authorization": "Bearer ${JIRA_TOKEN}"
      }
    }
  }
}
```

SIKKERHETSREGEL FOR MCP-SERVERE

En MCP-server arver *ikke* agentens tillatelsesgrenser automatisk — den har de rettighetene legitimasjonen du ga den, faktisk gir. Gi derfor alltid **lesetilgang som standard** og skrive tilgang bare der det er begrunnet, og pek utviklingsoppsett mot staging. En tredjeparts MCP-server er kode du kjører med dine rettigheter: behandle den som en avhengighet i forsyningskjeden, med de samme kravene til opphav og versjonslåsning.

22.2 Å skrive en egen MCP-server

De fleste organisasjoner trenger minst én egen server mot et internt system. Kvaliteten avgjøres nesten utelukkende av to ting: **verktøybeskrivelsene** og **feilmeldingene** — begge leses av en modell som skal ta en beslutning.

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";

const server = new McpServer({ name: "ordre", version: "1.2.0" });

server.registerTool("finn_ordre", {
  // Beskrivelsen er ikke dokumentasjon – den er det modellen velger verktøy ut fra.
  // Si hva verktøyet gjør, når det skal brukes, og hva det IKKE dekker.
  description: "Slår opp en ordre på ordrenummer eller kundens e-post. Returnerer status, " +
    "linjer og leveringsdato. Dekker ikke retur – bruk finn_retur til det.",
  inputSchema: {
    ordrenummer: z.string().optional(),
    epost: z.string().email().optional()
  },
  async ({ ordrenummer, epost }) => {
    if (!ordrenummer && !epost)
      return { isError: true, content: [{ type: "text",
        // Handlingsbar feil: si hva som mangler OG hva neste steg er.
        text: "Mangler søkekriterium. Oppgi ordrenummer (f.eks. ORD-10432) eller e-post." } ] };
    const r = await db.finnOrdre({ ordrenummer, epost });
    return { content: [{ type: "text", text: JSON.stringify(r) } ] };
  }
});
```

22.3 Agent SDK: egne agenter på samme motor

Der Claude Code er et ferdig produkt, gir Agent SDK samme verktøysløyfe som et bibliotek — med full kontroll over orkestrering, verktøytilgang og tillatelser. Det er riktig valg når agenten skal være en del av din egen tjeneste snarere enn et utviklerverktøy: en saksbehandlingsagent i en intern portal, en triage-agent bak et API, en planlagt jobb med egendefinert godkjenningsflyt.

TABELL 22.1 — NÅR DU VELGER HVA

Behov	Velg	Begrunnelse
Utviklere skal jobbe raskere	Claude Code	Ferdig produkt med IDE-, CLI- og CI-integrasjon. Ikke bygg dette selv
Automatisere en utviklingsarbeidsflyt	Claude Code i CI	<code>cClaude -p</code> med hooks og tillatelsesregler dekker de fleste tilfeller
Agent inn i eget produkt	Agent SDK	Du trenger kontroll på sesjon, autorisasjon per bruker og eget grensesnitt
Agenter på tvers av organisasjoner	Agent SDK + A2A	Ekstern samhandling krever oppdagelse, identitet og oppgavelivssyklus (kap. 9)

Verdt å merke seg: valget er ikke gjensidig utelukkende, og det vanligste modne oppsettet bruker begge — Claude Code som utviklerverktøy, Agent SDK for produktagenter, og **de samme MCP-serverne under begge**. Det er nettopp det kontraktlaget fra kapittel 4 gir: én verktøyimplementasjon, flere konsumenter.

Del IV · Kapittel 23

Produksjonssetting: CI, styring og kostnad

Overgangen fra «virker på min maskin» til «kjører uten tilsyn» krever tre ting som ikke trengs interaktivt: determinisme, stramme grenser og målbarhet.

23.1 I CI/CD

Den mest verdifulle automatiseringen er ikke kodegenerering, men **gjennomgang og triage** — arbeid som ellers står i kø fordi ingen har tid. Nøkkelen er å kjøre med minimale rettigheter og la mennesket beholde beslutningen.

```
# .github/workflows/gjennomgang.yml
on: { pull_request: { types: [opened, synchronize] } }
permissions: { contents: read, pull-requests: write } # minst mulig; ingen write på contents
jobs:
  gjennomgang:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
        with: { fetch-depth: 0 } # hele historikken, ellers blir diffen feil
      - run: curl -fsSL https://claude.ai/install.sh | bash
      - run: |
          git diff origin/${{ github.base_ref }}...HEAD --name-only \
          | claude -p "Gjennomgå disse filene for sikkerhetsproblemer og brudd på
          CLAUDE.md. Skriv kun funn du kan begrunne konkret."
          env: { ANTHROPIC_API_KEY: "${{ secrets.ANTHROPIC_API_KEY }}" }
```

23.2 Styringsmodell for organisasjonen

TABELL 23.1 — HVA SOM STYRES HVOR, OG HVEM SOM EIER DET

Kontroll	Sted	Eier	Eksempel
Obligatoriske sikkerhetskontroller	Managed policy settings	Sikkerhet	Blokkering av produksjonstilgang; påkrevd hemmelighetsskanning
Modell- og kostnadsgrenser	Managed settings + fakturering	Plattform	Tillatte modeller, forbrukstak per team
Prosjektkonvensjoner	.claude/ i repoet	Teamet	Formateringshook, testkommando, subagenter
Delte arbeidsflyter	Intern plugin-marketplace	Plattform + team	Utgivelsesprosedyre, gjennomgangsagent
Revisjonsspor	Hooks → sentral logg	Sikkerhet	SubagentStop- og SessionEnd-hooks som eksporterer hendelser

23.3 Kostnadsstyring

Kostnad i agentsystemer er drevet av tokens, og tokens er drevet av kontekst. Fire tiltak dekker det meste, i synkende rekkefølge etter effekt: **(1) delegér bredt arbeid til subagenter** — hovedkonteksten er den dyreste plassen å oppbevare noe; **(2) velg modell etter oppgave** — rutinemessig, avgrenset arbeid trenger ikke den største modellen, og `model:` -feltet i en subagent gjør dette til én linje; **(3) hold CLAUDE.md kort**, siden den betales hver sesjon; og **(4) utnytt caching** ved å holde den stabile delen av konteksten først og den varierende sist.

23.4 Målemetodikk: hva du faktisk skal rapportere

Spørsmålet «gir dette effekt?» besvares dårlig med aktivitetstall som antall sesjoner eller genererte linjer. Tre målinger holder, og de kan alle etableres på en dag:

- **Gjennomløpsti**d for en definert oppgavetype — for eksempel «fra issue åpnet til PR klar for gjennomgang» — målt før og etter innføring, på samme oppgavekategori.
- **Andel forslag** som går videre uten vesentlig omarbeiding. Dette er kvalitetsmålet; det fanger opp regresjoner som aktivitetstall skjuler fullstendig.
- **Kostnad per fullført oppgave**, ikke per kall. Nevneren skal være noe som har verdi for virksomheten, ellers optimaliserer man feil størrelse.

INNFØRINGSRÅD

Begynn med den arbeidsflyten som er *kjedeligst og best definert* — testskrivning, avhengighetsoppdateringer, utgivelsesnotater — ikke den mest ambisiøse. Godt definert arbeid gir raskt målbare resultater og bygger tilliten som trengs for å ta i bruk verktøyet på vanskeligere oppgaver. Team som starter med det mest ambisiøse, bruker som regel opp tålmodigheten før de har lært hvordan verktøyet oppfører seg.

Del V · Kapittel 24

Den tilgrensende stakken

Seks områder som ikke er hovedtema, men som avgjør om hovedtemaene lykkes. Hvert av dem er behandlet med den anbefalingen som følger av rapportens øvrige analyse.

24.1 Henting og kunnskapsgrunnlag (RAG)

Vektorsøk alene er ikke lenger tilstrekkelig som standardarkitektur. **Hybrid henting** — BM25 kombinert med vektorsøk, etterfulgt av en reranker — slår ren vektorlikhet konsistent, særlig på forkortelser og produktnavn der eksakt treff betyr noe. Nest største gevinst ligger i **oppdelingen**: chunking som følger dokumentets struktur slår fast tegnantall med god margin. En agent med filsystem- og søketilgang kan dessuten ofte hente det den trenger direkte — vurder alltid det før du bygger en indekseringspipeline.

24.2 Evaluering som disiplin

Evalueringsarbeid har den samme egenskapen som testing: det føles som en kostnad helt til det første gang forhindrer en regresjon i produksjon. Minimumsoppsettet er beskjedent — **50–200 reelle oppgaver med fasit, kjørt automatisk ved hver endring i prompt, modell eller verktøy**, med resultatet lagret over tid. Uten en tidsserie kan man ikke skille forbedring fra støy. Der fasit ikke finnes, gjelder forbeholdene om LLM-som-dommer fra kapittel 10.2: rubrikk, validering mot mennesker, og et kontrollsett utenfor optimaliseringen.

24.3 Isolering av kjøring

TABELL 24.1 — ISOLERINGSTEKNOLOGIER FOR AGENTKJØRING

Teknologi	Isolasjon	Oppstart	Bruk når
Container (namespaces)	Middels	ms	Standardvalget for klart kode med droppede kapabiliteter og nektet-nett
gVisor	Høy	ms	Uklart kode der syscall-flaten må reduseres uten full VM
Mikro-VM (Firecracker)	Svært høy	~100 ms	Flerleietaker, eller kjøring av kode agenten selv har skrevet
WASM / WASI	Høy	µs	Verktøyplugins med kapabilitetsbasert tilgang; økosystemet er fortsatt ujevnt

24.4 Forsyningskjede for agentutvidelser

Når agenter installerer plugins, skills og MCP-servere, er hver av dem kode som kjører med dine rettigheter. Løsningen er den etablerte: **signér med Sigstore, verifiser ved installasjon, lås versjoner, og krev SLSA-herkomst for det som kjører med skriverettigheter**. Et internt register med gjennomgåtte utvidelser er langt tryggere enn direkte installasjon fra offentlige kilder — og koster lite når signeringskjeden først er på plass.

24.5 Observabilitet og FinOps for AI

Rådet fra kapittel 2.2 gjentas fordi det er viktig: bruk OpenTelemetry med GenAI-konvensjonene, ikke en separat agentstakk. På kostnadssiden er den avgjørende disiplinen å **tilordne kostnad til noe forretningsmessig meningsfullt** — per team, per arbeidsflyt, per fullført oppgave. Kostnad per API-kall er en teknisk metrikk uten styringsverdi; kostnad per behandlet sak kan sammenlignes med alternativet og gir dermed en reell beslutning.

24.6 Prompt-injeksjon som systemrisiko

DEN VIKTIGSTE ENKELTRISIKOEN I HELE STAKKEN

Indirekte prompt-injeksjon — instruksjoner skjult i innhold agenten leser — har **ingen fullstendig løsning på modellnivå**, og det bør legges til grunn i arkitekturen framfor å håpes bort. Fire tiltak reduserer risikoen vesentlig, og de virker fordi de er uavhengige av modellens vurdering: **(1)** behandle alt hentet innhold som data, aldri som instruksjoner; **(2)** håndheve autorisasjon utenfor modellen, slik at en villedet agent ikke har rettighetene den blir bedt om å misbruke; **(3)** kreve menneskelig godkjenning for irreversible operasjoner; og **(4)** isolere lesing av uklart innhold i en subagent uten skrivende verktøy, slik SOC-eksempelet i kapittel 11.3 viser. Ingen av disse er promptbaserte — det er hele poenget.

24.7 Kontekstteknikk og modellvalg

«Prompt engineering» har flyttet seg fra formulering til **kontekstforvaltning**: hva som legges inn, i hvilken rekkefølge, og hva som holdes utenfor. Tre prinsipper er robuste på tvers av modeller: **stabilt først, variabelt sist** (gjør caching effektiv); **eksempler slår forklaringer** (to inn/ut-par styrer bedre enn et avsnitt med regler); og **verktøybeskrivelser er prompter** — ofte det mest leste og minst redigerte i hele systemet.

Modellvalg bør være en *kjøretidsbeslutning per oppgavetype*. Rutinearbeid løses av en mindre modell; planlegging og syntese trenger den største. I Claude Code er dette ett felt i en subagents frontmatter — blant de billigste kostnadsoptimaliseringene som finnes. Uten evalueringssett blir valget en smakssak, og da velges systematisk den dyreste.

Regulatorisk bilde og risikohåndtering

En teknisk lesning av rammeverket slik det står i august 2026, med vekt på det som faktisk påvirker arkitekturvalg. Dette er ikke juridisk rådgivning.

25.1 EU AI Act etter Digital Omnibus

Tidslinjen ble vesentlig endret i 2026: Digital Omnibus on AI ble vedtatt av Parlamentet 16. juni og av Rådet 29. juni 2026, med ikrafttredelse i juli. Endringen utsetter forpliktelsene for høyrisikosystemer, men *ikke* resten av regelverket.

TABELL 25.1 — GJELDENDE TIDSLINJE PER AUGUST 2026

Forpliktelse	Gjelder fra	Merknad
Forbudte praksiser	Februar 2025	Gjelder allerede. Uendret av omnibus
AI-kompetanse (art. 4)	Februar 2025	Beholdt på opprinnelig tidslinje
GPAI-modeller	August 2025	Gjelder allerede for leverandører av generelle modeller
Åpenhetsplikter (art. 50)	August 2026	Beholdt. Merking av AI-generert innhold og opplysning ved interaksjon
Høyrisiko, Annex III	Desember 2027	Utsatt fra 2. august 2026 — den store endringen
Høyrisiko, Annex I (innebygd i regulerte produkter)	August 2028	Utsatt fra august 2027

Utsettelsen ble begrunnet med at etterlevelsesekosystemet ikke var klart: harmoniserte standarder fra CEN-CENELEC lå etter, Kommisjonens veiledning forelå bare i utkast, og flere medlemsstater hadde ikke utpekt eller finansiert markedsovervåkingsmyndigheter. **Konsekvensen er tosidig:** reelt pusterom for høyrisikoarbeidet, men åpenhetspliktene i artikkel 50 gjelder *nå* — og de treffer bredere enn mange antar.

25.2 Personvern og agenter

GDPR stiller ingen nye krav til agenter spesielt, men agentarkitektur gjør fire eksisterende krav merkbart vanskeligere. **Formålsbegrensning:** en agent som selv velger kilder, kan utvide behandlingsformålet uten at noen har besluttet det — verktøytilgangen må avgrenses til formålet, ikke til hva som er teknisk mulig. **Dataminimering:** et kontekstvindu fylt med «alt som kan være relevant» er det motsatte av minimering. **Innsyn og sletting:** personopplysninger i erfaringsminne må kunne finnes igjen og slettes, hvilket forutsetter strukturert og sporbart minne. **Automatiserte avgjørelser:** treffer agenten beslutninger med rettslig virkning, gjelder artikkel 22, og menneskelig overprøving må være reell.

25.3 NIS2 og norsk kontekst

NIS2 kommer til norske virksomheter gjennom EØS-tilpasningen og treffer agentsystemer på tre punkter: **risikostyring i forsyningskjeden** (som omfatter MCP-servere, plugins og tredjepartsagenter, jf. kap. 24.4), **hendelsesrapportering** innen korte frister (som forutsetter at man kan rekonstruere hva systemet gjorde), og **ledelsesansvar**. I tillegg gjelder dataresidens, arkiv- og innsynskrav og sektorregelverk.

DET SOM GJØR ETTERLEVELSE ENKLEST

De tre tiltakene som gir mest regulatorisk uttelling, er allerede anbefalt av helt andre grunner: **fullstendig revisjonsspor** (kap. 4.1), **autorisasjon utenfor modellen** (kap. 4.2) og **menneskelig godkjenning for irreversible operasjoner** (kap. 24.6). Bygg for innsyn og kontroll, så blir dokumentasjonen i hovedsak å beskrive det som finnes.

25.4 Risikoregister

TABELL 25.2 — DE RISIKOENE SOM FAKTISK MATERIALISERER SEG, MED TILTAK OG EIER

Risiko	Sanns.	Konse- kvens	Tiltak	Eier
Indirekte prompt-injeksjon gjennom hentet innhold	Høy	Høy	Autorisasjon utenfor modellen; lesing av uklart innhold i subagent uten skriverettigheter (kap. 24.6)	Sikkerhet
Kostnadsoverskridelse ved ubudsjettet delegering	Høy	Middels	Budsjett i kode per delegering; varsling på hale i kostnadsfordelingen	Plattform
Kompromittert tredjeparts MCP-server eller plugin	Middels	Høy	Signering og verifisering, versjonslåsing, internt gjennomgått register (kap. 24.4)	Sikkerhet
Stille kvalitetsregresjon etter modell- eller prompting	Høy	Middels	Evalueringssett kjørt automatisk ved hver endring; tidsserie på resultatene	Produkt
Personopplysninger i erfaringsminne uten sporbarhet	Middels	Høy	Strukturert, gjennomgått minne med eier; sletting må være mulig (kap. 25.2)	Personvern

Konklusjoner og anbefalinger

26.1 Tolv anbefalinger, prioritert

TABELL 26.1 — ANBEFALINGER MED INNSATS, EFFEKT OG EIER

#	Anbefaling	Innsats	Effekt	Eier	Kapittel
1	Etabler MCP som kontraktgrense mot verktøy før den andre integrasjonen. Kostnaden ved å innføre grensen i etterkant vokser superlineært	Middels	Svært høy	Arkitektur	1.3, 4
2	Flytt all sikkerhetslogikk fra prompter til hooks og policy. En instruksjon er en anmodning; en hook er kode som kjører	Lav	Svært høy	Sikkerhet	4.2, 19.1, 20
3	Instrumentér med OpenTelemetry + GenAI semconv fra dag én, med kostnad per fullført oppgave som førsteklasses metrikk	Middels	Svært høy	Plattform	2.2, 10.3
4	Bygg enkeltagenten først og mål den. Innfør multi-agent kun der målingen viser en reell grense	Lav	Høy	Produkt	6
5	Gi hver delegering budsjett, tilstrekkelighetskriterium og stagnasjonsdeteksjon. Manglende terminering er den vanligste produksjonsfeilen	Middels	Høy	Utvikling	8.4, 10.1
6	Etabler et evalueringssett på 50–200 reelle oppgaver før første produksjonssetting, holdt utenfor utviklingssløyfen	Middels	Høy	Produkt	10.2, 24.2
7	Behold AV1 som standardvalg; gjør mediekjeden formatnøytral. Ingen AV2 i produksjon før maskinvarestøtten kommer	Lav	Middels	Media	13, 14.1
8	Innfør per-tittel bitrate-stige. 15–30 % båndbredde til en brøkdel av risikoen ved et formatbytte	Middels	Høy	Media	14.3
9	Standardisér på zstd med nivå per arbeidsmengde og ordbøker der objektene er små og likeartede	Lav	Middels	Plattform	17
10	Legg Claude Code-oppsettet i repoet, ikke i hjemmekataloger, og bruk managed settings for det som må gjelde alle	Lav	Høy	Plattform	18.3, 23.2
11	Signér og verifiser alle agentutvidelser; gi hver agent egen arbeidslasteridentitet	Middels	Høy	Sikkerhet	2.3, 24.4
12	Bruk pusterommet i AI Act-utsettelsen. Artikkel 50-åpenhetspliktene gjelder nå; høyrisikoarbeidet har fått tid	Middels	Middels	Juridisk	25.1

26.2 Veikart i tre horisonter

TABELL 26.2 — ANBEFALT REKKEFØLGE

Horisont	Gjør	Slik vet du at du er ferdig
0–3 mnd Grunnmur	Anbefaling 2, 3, 10. Telemetri, hooks, identitet, oppsett i repo. Formatnøytral mediekjede (7)	Et vilkårlig verktøykall kan spores ende-til-ende og knyttes til identitet og godkjenning
3–9 mnd Første verdi	Anbefaling 1, 4, 6, 8, 9. Én MCP-server, én agent i drift med evaluering, per-tittel-stige, zstd	Suksessrate og kostnad per kjøring er kjent og stabil over 200+ reelle kjøring
9–24 mnd Bredde og samhandling	Anbefaling 5, 11, 12. Flere arbeidsflyter, plugin-distribusjon, A2A utad, AI Act-arbeid	Ny arbeidsflyt går i drift uten endring i grunnmuren; marginalkostnaden faller

26.3 Sluttbemerkning

De fire områdene i mandatet konvergerer mot den samme innsikten, og den er verdt å gjenta til slutt fordi den er lett å nikke til og vanskelig å etterleve: **verdien ligger i grensesnittet, ikke i implementasjonen**. En codec er en kontrakt som overlever både koderen og dekodeen som implementerer den. MCP og A2A er den samme typen kontrakt for agentsystemer. Claude Codes primitiver er kontrakter mellom menneske og agent. I alle tre tilfellene er den dyre feilen den samme — å optimalisere implementasjonen mens grensesnittet forblir implisitt — og den billige seieren er også den samme: gjør kontrakten eksplisitt, testbar og versjonert, og la implementasjonene bak den være utskiftbare.

Det følger en praktisk konsekvens av dette som går på tvers av alle anbefalingene over. Teknologiene i denne rapporten vil endre seg raskere enn arkitekturen som bruker dem: AV2 får maskinvare, MCP får nye versjoner, modeller blir bedre og billigere, og regelverket flytter seg igjen. Det som avgjør om organisasjonen kan ta imot de endringene billig, er ikke hvilke valg som tas i dag, men **hvor eksplisitte grensene mellom komponentene er**. Bygg for utskiftbarhet, mål det du styrer etter, og hold kontrollene utenfor modellen — så blir resten konfigurasjon.

Vedlegg

Ordliste, kilder, sjekklister og metode

Vedlegg A · Ordliste

- **A2A** — Agent2Agent. Åpen protokoll for samhandling mellom uavhengige agenter; hostet av Linux Foundation.
 - **Agent Card** — Signert JSON-dokument som beskriver en A2A-agents kapabiliteter, endepunkt og autentisering.
 - **BD-rate** — Bjøntegaard-Delta-rate. Gjennomsnittlig bitrateforskjell ved lik kvalitet mellom to enkodere.
 - **CIMD** — Client ID Metadata Documents. Erstatte dynamisk klientregistrering i MCPs autorisasjonsprofil.
 - **GOP** — Group of Pictures. Avstand mellom nøkkelbilder; bør være multiplum av segmentlengden.
 - **Hook** — Deterministisk kommando eller kall som kjøres ved en livssyklusendelse i Claude Code.
 - **MCP** — Model Context Protocol. Åpen standard for tilgang fra modell til verktøy, ressurser og prompter.
- **MRTR** — Multi Round-Trip Requests. Statsløs mekanisme i MCP for avklaringer midt i et verktøykall.
 - **RVQ** — Residual vektorkvantisering. Kaskade av kodebøker der hvert nivå koder foregående nivå's feil.
 - **SLSA** — Rammeverk for verifiserbar herkomst i forsyningskjeden for programvare.
 - **SPIFFE** — Standard for kryptografisk arbeidslastidentitet uten delte hemmeligheter.
 - **Subagent** — Delegert agentkjøring med egen kontekst, egne verktøygrenser og egne tillatelser.
 - **VMAF** — Lært videokvalitetsmetrikk kalibrert mot menneskelige vurderinger.
 - **xHE-AAC** — AAC-variant basert på USAC; dekker hele bitratespennet fra tale til musikk.

Vedlegg B · Kildegrunnlag

TABELL B.1 — PRIMÆRKILDER FOR TALL OG PÅSTANDER SOM ER KILDEMERKET I TEKSTEN

Tema	Kilde	Brukt til
MCP-spesifikasjon	modelcontextprotocol.io — spesifikasjon 2026-07-28 og tilhørende utgivelsesnotat	Kapittel 3 i sin helhet: statsløs kjerne, MRTR, SEP-numre, avviklingspolicy
A2A-status	Linux Foundation og Google Open Source, ettårsmarkering april 2026	150+ organisasjoner, 1.0/1.0.1, skyleverandørintegrasjon (kap. 2, 9)
AV2	AOMedia, spesifikasjon v1.0.0 (28. mai 2026) og tilhørende dekning	~30 % mot AV1, utslag på skjerminnhold/HDR, maskinvare 2027–28 (kap. 13, 14)
Multi-agent tokenøkonomi	Publiserte driftserfaringer fra produksjonssatt forskningsagent	4× og 15× tokenforhold; ~80 % av ytelsesvarians (kap. 6.2)
Claude Code	code.claude.com/docs — oversikt, hooks-referanse, subagenter	Kapittel 18–23: overflater, hendelsestabell, JSON-kontrakt, konfigurasjonshierarki
Lyd- og nevrale codecs	Publiserte codec-sammenligninger og modellenes egne spesifikasjoner	Kapittel 15–16: bitrater, rammerater, arkitektur
EU AI Act	Digital Omnibus on AI — Parlamentet 16.6.2026, Rådet 29.6.2026	Utsatte frister til desember 2027 / august 2028 (kap. 25.1)

Tall uten kildemerking i teksten er rapportens egne estimater, angitt som størrelsesordener. Kompresjons- og ytelsestall varierer med implementasjon, innstillinger og materiale, og skal brukes til å rangere alternativer — ikke til kapasitetsplanlegging.

Vedlegg C · Sjekklister

Før første agent i produksjon

- Telemetri med kostnad per fullført oppgave
- Egen identitet per agent
- Idempotens og angreoperasjon på alt som skriver
- Budsjett håndhevet i kode, ikke i prompt
- Evalueringssett utenfor utviklingsløyfen
- Sikkerhetskontroller i hooks, ikke i instruksjoner

Før formatbytte i mediekjeden

- 20–50 representative klipp målt på eget materiale
- Sammenlignbart beregningsbudsjett mellom kandidatene
- Metrikk oppgitt eksplisitt i alle sammenligninger
- Maskinvaredekoding verifisert i målgruppens utstyr
- Kvalitet målt mot kilden, ikke forrige generasjon
- GOP festet til segmentlengden

Vedlegg D · Metode

Rapporten bygger på primærspesifikasjoner, leverandørdokumentasjon og publisert benchmarklitteratur, verifisert per juli–august 2026. Der kilder spriker, er dette angitt eksplisitt framfor å velge én versjon; der grunnlaget ikke holder til en anbefaling, er det sagt. Anbefalingene i kapittel 26 er rangert etter forholdet mellom forventet effekt og innsats, med reversibilitet som tiebreaker: tiltak som er lette å angre, prioriteres foran tiltak med samme forventede effekt men høyere byttekostnad.